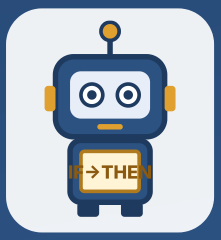
The SilkMind Caravan

Old routes. New roots.

A hands-on study guide to **Software & Symbolic AI**: travel the Silk Road with a robot who follows rules, a parrot who learns from examples, and a navigator who explains what is really going on.



BEK


To'ti



Navigator Nodira

Study Guide V1.0 · Stage 2 · November 2026
Team **SilkMind**: Afandiyeva Hayat · Khaydarov Sarvarbek · Nabizadehshahrbabak Arsalan · Egemberdiyeva Aliya · Kenzhebolat Anel · Mukhtarov Mukhammadshakhruz · Bakatov Madi
Module: Software & Symbolic AI · Built from Markdown Content V2.0 · github.com/skh0035/Mini-AI-repository
Companion files: Test Bank (Word) · lab files (.pl, .py, .ipynb) · printable cards (Appendix A)

Contents

Welcome aboard the caravan 3

What you need · How every stop works 4

Your route · learning goals · choose your route 5

Stop 1 · Khiva: The Gate That Says No 6

Mission 1.1 Be BEK: the Exact Instructions game 6

Mission 1.2 The Gatekeeper's rulebook 8

Mission 1.3 Keep the caravanserai warm 9

Stop 2 · Bukhara: BEK's Book of Rules 12

Mission 2.1 Rule Cards: be the inference engine 13

Mission 2.2 Twenty Questions, rule-style 15

Lab 2A BEK's rules in real Prolog (SWISH) 15

Lab 2B ELIZA: rules that pretend to understand 16

Mission 2.3 Explain like the scholar 18

Stop 3 · Samarkand: To'ti's Market 22

Mission 3.1 Teach To'ti: learn a rule from data 22

Lab 3 Train a scam detector (Machine Learning for Kids) 24

Mission 3.2 BEK or To'ti? Choose the right tool 26

Stop 4 · Otrar: The Frozen Caravanserai 29

Mission 4.1 Timeline cards: boom or winter? 29

Mission 4.2 Crack the lock 30

Mission 4.3 The fact-checker's desk 31

Mission 4.4 It's 1985: pitch an expert system 32

Stop 5 · Kashgar: The Modern Bazaar 35

Mission 5.1 Rule hunt: forms and apps 35

Mission 5.2 Guard duty 36

Mission 5.3 The delivery-fee bug 37

Lab 5 The Silk Road knowledge graph (SWISH) 38

The Grand Caravan Challenge 42

Glossary 43

Appendix A · Printable cards 45

Appendix B · How we tested this guide 47

Resources, references and acknowledgements 48

Welcome aboard the caravan



NAVIGATOR NODIRA

Salom! I'm Nodira, navigator of this caravan. Over five stops you will **build, break and fix** small rule-based systems, train a model, and find out why the world twice stopped believing in rule-based AI. You will spend more time *doing* than reading.



BEK THE RULE-ROBOT

I am BEK. I follow rules. Exactly. Every time. If the rule is wrong, I am wrong. Please write good rules.



TO'TI THE PARROT

And I'm To'ti! Nobody gives me rules: I copy patterns from examples. Usually that works. Sometimes... squawk.

Who this guide is for

First-year students from **any** subject with **no** programming, computer-science or AI background. If you can use a phone, read a table and compare two numbers (is 15 smaller than 18?), you are ready.

What you need

For	You need	If you can't get it
Every stop	Pen, paper, this guide, a partner if possible	Work alone: play every role yourself
Card missions (Stops 2–4)	Printed and cut cards from **Appendix A** (or copy them onto paper)	Use the tables in the mission instead of cards
Labs 2A, 5 (Prolog)	A browser and swish.swi-prolog.org. No account or install needed. Starter files are in `supporting-files/lab-files/`.	Use the screenshots and outputs printed in the lab, or run `supporting-files/examples/topic02_mini_rule_engine.py`
Lab 2B (ELIZA)	A browser and masswerk.at/elizabot	Analyse the printed transcript in the lab
Lab 3 (Machine Learning for Kids)	A browser, machinelearningforkids.co.uk → "Try it now" (no account; you must tick a "Verify you are human" box)	Run `supporting-files/examples/topic03_learn_a_rule.py` or the notebook `SilkMind_Labs.ipynb`
Optional code	Python 3 (no extra packages) or Google Colab / Jupyter for the notebook	Every result is also printed in this guide

Access note: our testing record is in Appendix E. Some sites (YouTube in particular) are not reachable on every network, so every video and lab has a written summary, screenshots or a fallback, and you can always continue without it.

How every stop works

Each stop is built around **one question** and follows the same rhythm:

Meet a problem → **predict** → **try** it → **observe** → understand the **real thing** → **apply** it → **check**

🐪 **CARAVAN TALE** Story and analogy. Fun, but not a technical claim.

🧭 **THE REAL THING** The accurate technical explanation. This is what you are tested on.

🔮 **PREDICT FIRST** Write a guess *before* you try. Wrong guesses teach the most.

✅ **CHECKPOINT** Quick questions; answers in the Feedback Corner at the end of the stop.

M Mission

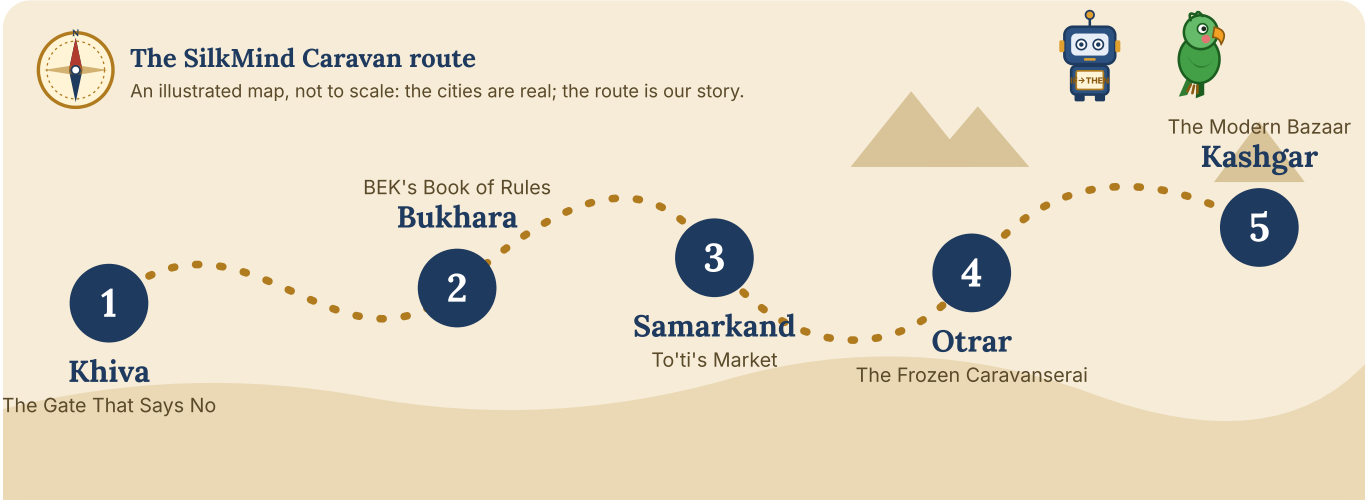
6 PARTS	Goal · Preparation · Task · Observation · Reflection · Feedback

🧪 **LAB** A real online tool, with exact steps, screenshots and a fallback.

🎬 **WATCH / EXPLORE** A short video or page, with what to watch for.

⚠️ **TRAPS** Mistakes beginners often make, and the truth.

Your route



What you will be able to do

#	By the end you can…	Stop
LO1	Explain what software, an algorithm and a Boolean condition are, using an everyday example.	1
LO2	Write a decision as an IF–THEN rule and check it with a truth table.	1
LO3	Describe the parts of a rule-based (symbolic) AI system: facts, rules, knowledge base, working memory, inference engine.	2
LO4	Trace forward and backward chaining by hand, and in Prolog, and explain a conclusion.	2

LO5	Contrast symbolic AI with machine learning, trace how a simple model learns a rule from data, and choose an approach.	3
LO6	Explain why symbolic AI rose, fell (the "AI winters") and changed, using evidence.	4
LO7	Find and trace rule-based reasoning in modern software, including systems that mix rules and learning.	5

Choose your route

Route	What to do	Time
Full caravan (recommended)	Every mission, lab and checkpoint, in order, then the Grand Caravan Challenge	≈ 5 h, over 2–3 sessions
Express route	Missions 1.2, 2.1, 3.1, 4.1, 5.2 + all checkpoints + Lab 2A	≈ 2.5 h
Lab lover	Express route + Labs 2B, 3, 5 + the Grand Challenge in SWISH	≈ 4 h

Stops must be taken **in order**: each one uses what you discovered at the stop before. The test bank (separate document) checks every learning outcome.

STOP 1 · KHIVA

The Gate That Says No



How does a machine decide who may pass, and why does it never "get the idea"?

Markdown source: `01-software-and-logic.md` · Outcomes LO1, LO2 · ≈ 45 min · Missions 1.1–1.3

🐫 CARAVAN TALE (STORY, NOT A TECHNICAL CLAIM)

The caravan reaches Khiva, whose old inner town, Itchan Kala, is still surrounded by tall clay walls. The guards are tired, so the city hires **BEK** as a gatekeeper. BEK is given one sheet of instructions and told: *"Let in only people who belong here."*

On his first morning, BEK lets in a goat (nobody said goats were not allowed), refuses the mayor (he forgot his pass), and spends twenty minutes trying to "check the weather" because the sheet said *"close the gate if it looks bad outside."*



BEK THE RULE-ROBOT
I did exactly what the sheet said. Is that not what a good machine does?



NAVIGATOR NODIRA
It is! That is the whole point of this stop: a computer does **exactly** what its instructions say, nothing more. So the instructions must be perfect.

🔮 PREDICT FIRST

Before you start, write down your guess: **why** did BEK spend twenty minutes on *"close the gate if it looks bad outside"*? What would you change in that instruction so a machine could follow it?

1.1 Be BEK: the Exact Instructions game

⏱ 15 min

GOAL	Discover that computers follow instructions **literally**, and that a good instruction is precise, ordered and complete (an **algorithm**).
PREPARATION	A partner, two sheets of paper, a pen. No prior knowledge.

TASK	1. **Writer** (back-to-back with BEK, so BEK cannot see): draw a simple *caravan tent*: a square with a triangle on top and a door. Hide it. 2. The Writer writes **at most 8 instructions** so BEK can draw the same picture without seeing it. 3. **BEK** follows each instruction **exactly as written**. If it is unclear, BEK does the strangest thing it could mean, and does **not** ask questions. 4. Compare the two drawings. Then swap roles and try again with improved instructions.
OBSERVATION	Record in a small table: *instruction number · what the Writer meant · what BEK did*. Mark each instruction that failed. Count: how many failed in round 1? In round 2?
REFLECTION	Which words caused most failures (e.g. "small", "near", "a bit")? What did you add in round 2 to make instructions work: numbers? order? a stop condition?
FEEDBACK	Hint: look for missing **start points**, **sizes** and **order**. Explained outcome → Feedback Corner, Mission 1.1.

🎬 WATCH / EXPLORE

Exact Instructions Challenge – THIS is why my kids hate me by Josh Darnit (YouTube)
https://www.youtube.com/watch?v=cDA3_5982h8

Why use it?	A father makes a sandwich by following his children's written instructions *literally*. It is the same game you just played, and very funny.
Which part?	Watch the whole video (it is short).
Your task	Pause after each instruction the children give. Predict what Dad will do. Then list two instructions you would rewrite, and how.
Watch out	Dad is pretending to be a machine for comedy; real computers don't "choose" the silliest meaning. They simply cannot fill gaps at all. Not available? Mission 1.1 teaches the same idea.

🧭 THE REAL THING: SOFTWARE, ALGORITHMS AND CONDITIONS

Software is the set of instructions (programs) that tell a computer what to do; **hardware** is the physical machine: screen, processor, sensors, keyboard. You cannot touch software. (Britannica: Software)

An **algorithm** is a step-by-step procedure that solves a problem in a finite number of clear steps. A recipe is an algorithm for people; a program is an algorithm a computer can run. (Britannica: Algorithm)

Many steps are **decisions**, such as *"IF the password is correct THEN let the user in, ELSE refuse."* The question "is the password correct?" has only two answers, **true** or **false**. It is a **Boolean condition**, named after George Boole, who set out this logic in 1847. (Britannica: Boolean algebra) Conditions are combined with three operators:

Operator	True when…	Example
AND	**both** parts are true	account exists AND password matches
OR	**at least one** part is true (including both)	is a student OR is over 65
NOT	flips true ↔ false	NOT account is locked

A **truth table** lists every combination of true/false inputs and the result for each, so no case is forgotten.

1.2 The Gatekeeper's rulebook (login logic)

⏱ 15 min

GOAL

Trace a real decision (an app's login) step by step, check it with a truth table, and **find the error** in a faulty rule.

PREPARATION

The Real Thing box above. Optional: two cards marked **T** and two marked **F**.

TASK

Part A: trace. An email app receives an email address and a password. It checks: **(1)** does an account exist for this email? **(2)** does the password match? Only if both are true does it open the inbox. Follow the flowchart below for these three visitors and write the result:

Visitor	Account exists?	Password matches?	Result?
Aziz	True	True	
Bekzod	True	False	
A stranger	False	–	

Part B: find the error. A trainee writes the rule as `access = account_exists OR password_matches`. Lay your T/F cards out in all four combinations and fill in a truth table. Find the row where **the wrong person gets in**. Fix the rule.

OBSERVATION

Write the truth table for the trainee's OR rule and circle every row whose result is dangerous.

REFLECTION

Why do both failure paths in the flowchart show the *same* message, "Invalid email or password"? (Think like an attacker.)

FEEDBACK

Hint: OR is true when at least one part is true. Answers → Feedback Corner, Mission 1.2.

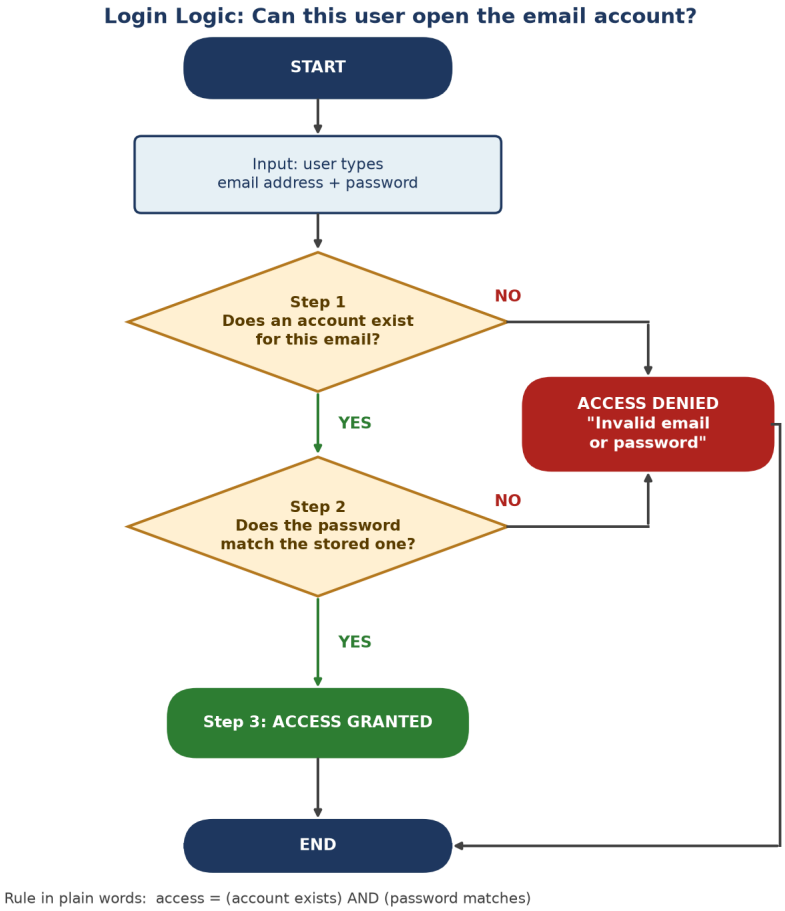


Figure 1.1 The login decision. Each diamond is one Boolean test with exactly two exits. Only YES + YES reaches "Access granted".

1.3 **Keep the caravanserai warm (write your own rule)**	⏱ 15 min
GOAL	Write a complete algorithm containing one IF–THEN–ELSE rule, and test its **boundary**.
PREPARATION	Mission 1.2 done. Pen and paper.
TASK	The caravanserai has a smart thermostat with a temperature sensor and a heater switch. 1. **Task A:** write a **3-step algorithm** that turns the heater ON when the temperature is **below 18 °C**: (1) get the input, (2) test a true/false condition, (3) act on the result with **one IF–THEN–ELSE rule**. 2. **Task B (predict, then check):** what does your rule do at 15 °C, **18 °C** and 21 °C? 3. **Task C (stretch):** change the rule so the heater turns on only if it is below 18 °C **and** someone is staying in the room. Write its truth table.
OBSERVATION	Fill in: *reading · condition true/false · heater* for 15, 18, 21 °C.
REFLECTION	Did anyone in your group get 18 °C "wrong"? What does that tell you about the word "below"?
FEEDBACK	Hint: "below 18" means strictly less than 18. Answers → Feedback Corner, Mission 1.3.

⚠️ TRAPS ON THE ROAD (COMMON MISUNDERSTANDINGS)

Trap	The truth
"OR means one or the other, not both."	In Boolean logic OR is true when **at least one** part is true, including both.
"The computer understands what a password is."	It only compares two stored pieces of data and gets true or false. Meaning comes from the people who wrote the rules.
"Pseudocode is a programming language."	Pseudocode is plain-language steps used to plan an algorithm. It cannot be run.
"Close enough is fine for a computer."	"Below 18" and "18 or below" are different rules. One character changes the behaviour.

✅ CHECKPOINT (ANSWERS IN THE FEEDBACK CORNER)

1. Is a phone's camera app hardware or software? The lens?
2. Why is "turn the heater on if it's chilly" a bad instruction for a computer?
3. `has_ticket = True`, `ticket_expired = True`. What is `has_ticket AND NOT ticket_expired`?
4. A café gives a discount to students **or** people over 65. Does a 70-year-old student get it?

🗝️ Feedback Corner

Try everything first. Then check here: every answer explains *why*.

Predict box. "Looks bad" is not a true/false condition. BEK cannot measure "bad". A machine-ready version: `IF wind_speed > 40 km/h THEN close gate`.

Mission 1.1 (expected outcome). Most first-round failures come from missing **sizes** ("draw a square": how big?), **positions** ("put a triangle on top": where exactly?), **order**, and vague words. Second-round instructions usually work better because they add numbers, a start point and a clear order. That is what an **algorithm** is: finite, clear, ordered steps. A computer, like BEK, cannot fill gaps.

Mission 1.2. Part A: Aziz → granted; Bekzod → denied; stranger → denied (step 1 fails, so step 2 is never reached). Part B: the trainee's OR rule gives **True** when the account exists but the password is wrong: **anyone who types a real user's email gets in with any password.** Correct rule: `access = account_exists AND password_matches`. Reflection: one generic message stops attackers from learning which email addresses have accounts; this is recommended practice (OWASP Authentication Cheat Sheet).

Mission 1.3. Task A: `T = read sensor` → test `T < 18` → `IF T < 18 THEN heater = ON ELSE heater = OFF`. Task B: 15 °C → ON; **18 °C → OFF** (18 is not less than 18); 21 °C → OFF. Writing `T <= 18` changes the rule. Task C: `IF (T < 18) AND (someone_staying) THEN ON ELSE OFF`; the only ON row is True/True. Common mistakes: skipping the input step; vague words such as "cold"; forgetting the ELSE.

Checkpoint. 1: app = software, lens = hardware. 2: "chilly" has no exact true/false meaning; use `T < 18`. 3: `NOT True` = False, so `True AND False` = **False**. 4: **Yes**: OR is true when at least one part is true (here both).

📮 **POSTCARD FROM THIS STOP**

Greetings from Khiva! BEK learned that a machine does **exactly** what it is told. Decisions are **Boolean conditions** (true/false) joined with **AND / OR / NOT**, written as **IF–THEN–ELSE rules** and checked with **truth tables**. Next stop: what happens when you give BEK *hundreds* of rules?

STOP 2 · BUKHARA

BEK's Book of Rules



Can a machine act like an expert if all it does is follow rules?

Markdown source: `02-rules-and-inference.md` · Outcomes LO3, LO4 · ≈ 80 min · Missions 2.1–2.3 · Labs 2A (Prolog), 2B (ELIZA)

🐫 CARAVAN TALE (STORY, NOT A TECHNICAL CLAIM)

Bukhara has been a city of scholars for over a thousand years; the great physician Ibn Sina was born nearby. In an old library, a scholar hands BEK a notebook: *"Travellers describe strange animals. Use my rules to tell them what they saw."* The notebook holds six rules. BEK cannot see the animals; he only hears descriptions.

Rule	IF ...	THEN ...
R1	has feathers	is a bird
R2	gives milk	is a mammal
R3	is a bird AND cannot fly AND swims	is a penguin
R4	is a bird AND can fly AND hunts at night	is an owl
R5	is a mammal AND eats meat AND has black stripes	is a tiger
R6	is a mammal AND has hooves AND has black stripes	is a zebra



BEK THE RULE-ROBOT

A traveller says: *it has feathers, it cannot fly, it swims*. No rule says "penguin" from those words directly. Must I give up?



NAVIGATOR NODIRA

No: you can **chain** rules. One rule's conclusion can become another rule's condition.

🔮 PREDICT FIRST

Using only R1–R6, what will BEK conclude from *has feathers, cannot fly, swims*? How many rules will he need to use, and in what order?

🧭 THE REAL THING: THE PARTS OF A RULE-BASED SYSTEM

BEK is now an **expert system**: a program that solves problems in one specialised area, the kind usually needing a human expert, using a **knowledge base** of rules and an **inference engine** (Britannica: Expert systems).

Part	What it is	In BEK's library
Symbol	A label that stands for something. The computer does **not** know what it means.	`has feathers`
Fact	A statement taken as true for the current case	"cannot fly"
Rule	IF (all conditions true) THEN (add a conclusion)	R1–R6
Knowledge base	All the rules; **the same** for every case	The scholar's notebook
Working memory	The facts for **this** case: starting facts + new ones	BEK's notepad
Inference engine	The loop that finds rules whose IF part is true and **fires** them	BEK's "brain"

Figure 2.1 Parts of a rule-based (symbolic) AI system

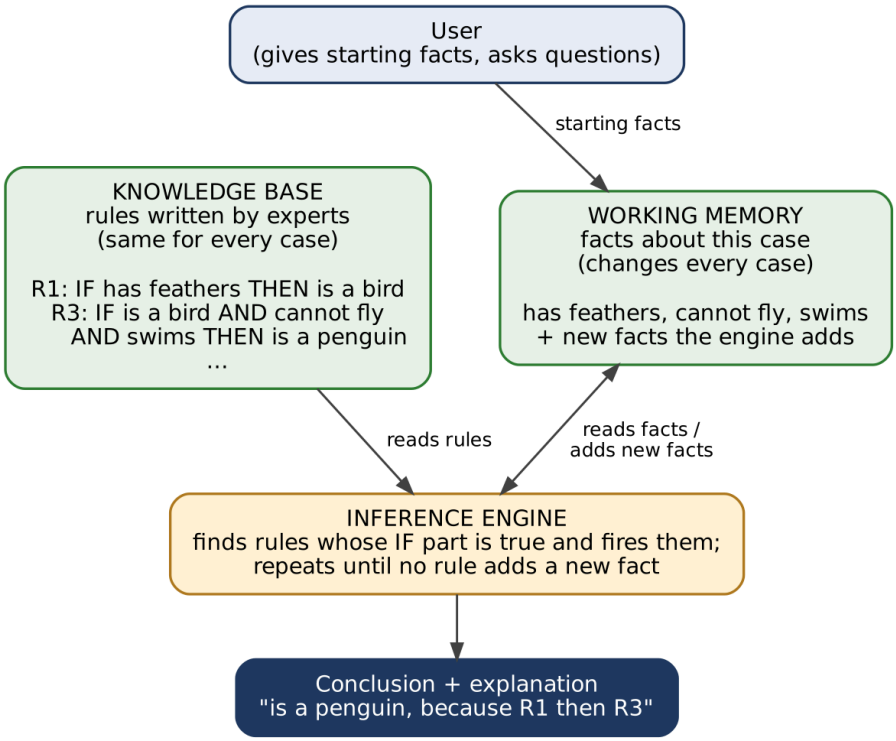


Figure 2.1 Parts of a rule-based system. Only the inference engine "does" anything; the knowledge base stays fixed while working memory changes for each case.

2.1 Rule Cards: be the inference engine

⏱ 20 min

GOAL

Experience **forward chaining** from the inside: fire rules cycle by cycle, record a **trace table**, and see when the engine stops.

PREPARATION

Rule cards R1–R6 and fact cards (Appendix A), or the table above. Groups of 3: **Memory keeper** (holds fact cards), **Engine** (checks rules), **Recorder**.

TASK

1. Memory keeper lays down the starting facts: **has feathers · cannot fly · swims**.
2. Engine checks the rule cards **in order R1 → R6**. The first rule whose IF part is **completely** on the table **and** whose THEN fact is not yet on the table **fires**: the Memory keeper adds its THEN card.
3. That is one **cycle**. Start again from R1. Stop when a whole cycle adds nothing new.
4. Play again with: **gives milk · has hooves · has black stripes**.
5. Last round: **gives milk · eats meat · has hooves · has black stripes**. Predict the result before you play!

OBSERVATION

Recorder fills a trace table for each round:

Cycle	Facts on table at start	Rule fired	New fact
1			
2			
3			
4			

REFLECTION

Why can't R3 fire in cycle 1 of round 1? What went strange in the last round: is the **engine** wrong, or the **rules**?

FEEDBACK

Hint: new facts only become usable in the *next* cycle. Answers → Feedback Corner, Mission 2.1.

Figure 2.2 Forward chaining: facts → rules → new facts
Blue = starting facts Yellow = rule that fires Green = fact added by the engine

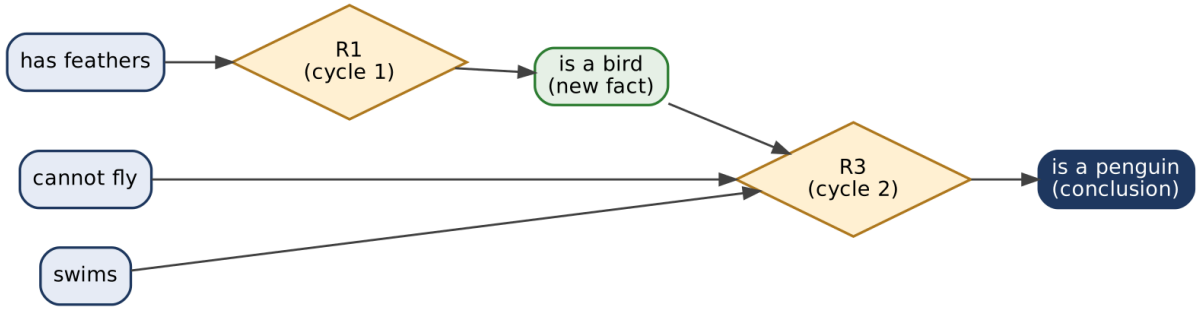


Figure 2.2 Round 1 as a picture: R1 turns "has feathers" into "is a bird"; then R3 combines three facts into "is a penguin".

🧭 THE REAL THING: FORWARD CHAINING, BACKWARD CHAINING AND EXPLANATIONS

Forward chaining starts from facts and fires rules until nothing new can be added. It is **data-driven**. **Backward chaining** starts from a **goal** ("is it a penguin?") and works backwards to the facts needed to prove it. It is **goal-driven**, and it can **ask** the user only the questions that matter. (Wikipedia: Forward chaining)

Because every new fact was added by a named rule, the system can **explain** itself: *"penguin, because R3; bird, because R1; the rest you told me."* Real systems such as MYCIN could explain their reasoning on request (Britannica: Expert systems).

2.2 Twenty Questions, rule-style (backward chaining) ⏱ 10 min

GOAL	Use backward chaining to ask only the questions needed to prove or reject a goal.
PREPARATION	R1–R6. A partner secretly picks one of the animal descriptions from Mission 2.1.
TASK	1. Choose a goal, for example **"is a tiger?"**. Find the only rule that concludes it (R5) and list its conditions. 2. For each condition that is itself a conclusion (*is a mammal*), find *its* rule (R2) and list *its* conditions. 3. Now ask your partner **only** those questions, in order. Stop as soon as one answer is "no".
OBSERVATION	Write your question list for the goal "tiger". Count how many questions you needed. How many would "ask about everything" need?
REFLECTION	Why would a doctor's diagnosis system prefer backward chaining?
FEEDBACK	Answers → Feedback Corner, Mission 2.2.

🧪 LAB: 2A · BEK'S RULES IN REAL PROLOG (SWISH)

Goal. Run the same six rules in **Prolog**, a real programming language for logic, predict its answers, add a rule, and investigate surprising results. **Platform:** SWISH (SWI-Prolog in the browser), swish.swi-prolog.org. Free, no account. Tested 3 Nov 2026 (Appendix E). **File:** `supporting-files/lab-files/lab2_animal_expert.pl`.

Just enough Prolog

You write	Read it as
`fact(pingu, swims).`	A **fact**: "Pingu swims." Every line ends with a full stop.
`is_a(X, bird) :- fact(X, has_feathers).`	A **rule**: "X is a bird **IF** X has feathers." `:-` means IF; a comma means AND.
`X`, `What`, `Who`	Words starting with a **capital letter** are **variables** ("something"). Lower-case words are fixed names.
`is_a(pingu, What).`	A **query**: "What is Pingu?" Prolog answers by searching the rules.
`% text`	A comment, ignored by Prolog.

Steps

1. Open SWISH → click **Program** (top of the left panel). Delete anything in the editor.
2. Paste the whole of `lab2_animal_expert.pl` into the left editor.
3. Type a query in the bottom-right box (*Your query goes here...*), e.g. `is_a(pingu, What).`, and press **Run!**
4. Prolog shows the first answer. Click **Next** for more, or **Stop** when done. *Tip: if SWISH says "Too many open queries", press Stop on old queries.*

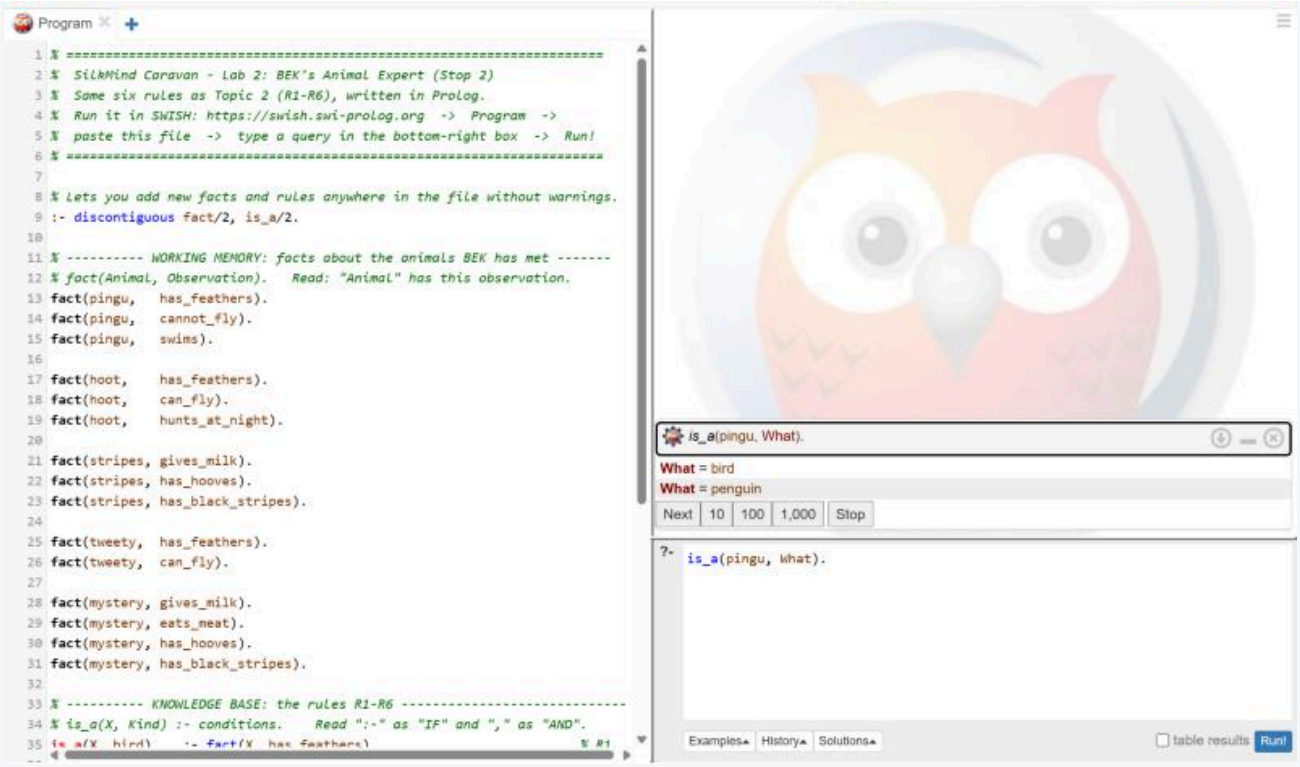


Figure L2.1 Our test run in SWISH: the program on the left; the query `is_a(pingu, What).` gave `What = bird`, then (after Next) `What = penguin`.

Investigate (predict first, then run)

#	Query	Your prediction	Prolog's answer
1	`is_a(pingu, penguin).`		
2	`is_a(Who, bird).` (press Next until it stops)		
3	`is_a(tweety, What).`		
4	`is_a(mystery, What).`		
5	`is_a(Pingu, penguin).` (capital P!)		

Build: add a giraffe called `gerald`. Add three facts (gives milk, has a long neck, has hooves) and a rule R7 that concludes `giraffe`. Test with `is_a(gerald, What).` Then deliberately misspell one fact (e.g. `has_long_nek`) and run it again.

Reflect: Prolog answers `is_a(pingu, penguin)` by starting from the **goal**. Which kind of chaining is that? Why did Tweety get only one answer?

Fallback (no internet): read the outputs in the Feedback Corner and Figure L2.2, or run `supporting-files/examples/topic02_mini_rule_engine.py` (forward chaining in Python, same rules).

🧪 LAB: 2B · ELIZA: RULES THAT *PRETEND* TO UNDERSTAND

Goal. Discover how a famous 1966 program held "conversations" using nothing but **pattern rules**, and find where the illusion breaks. **Platform:** ELIZA in JavaScript by Norbert Landsteiner (2005), a re-creation of Joseph Weizenbaum's 1966 DOCTOR script: masswerk.at/elizabot/eliza.html → click

open ELIZA terminal. No account. (Other versions, including the reconstructed original listed by the ELIZA history project at elizagen.org, may answer differently.)

Steps

1. Open the terminal and type each of the seven test lines below, pressing Enter after each. Use your own words afterwards.
2. Record ELIZA's reply next to each line, and **circle the word** in your input that you think triggered it.

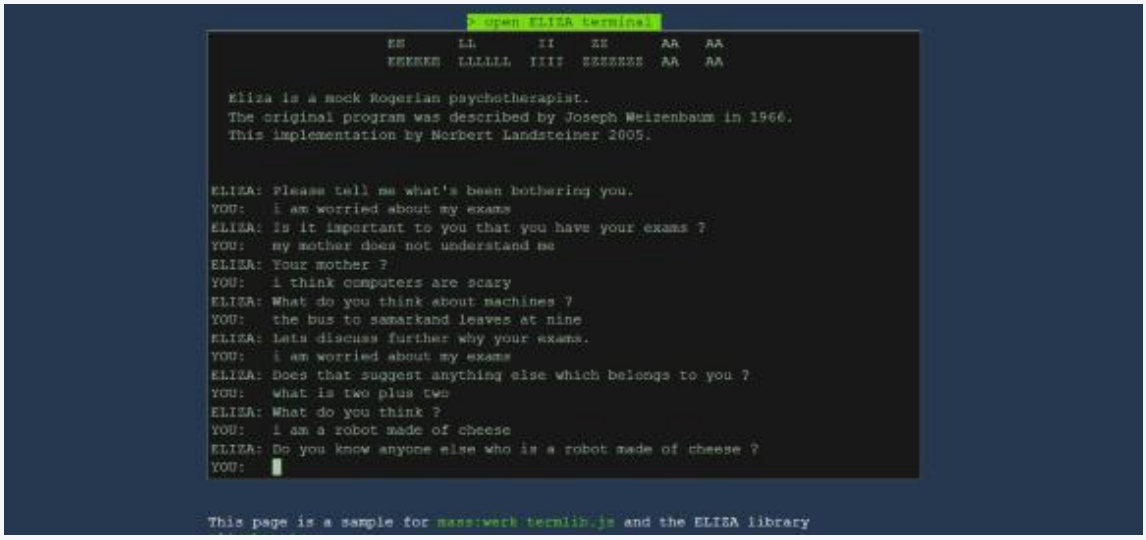


Figure L2.3 Our test conversation (3 Nov 2026). In the version we tested, ELIZA's replies can vary between runs; even its opening line changed when we reloaded.

#	We typed	ELIZA replied
1	i am worried about my exams	Is it important to you that you have your exams ?
2	my mother does not understand me	Your mother ?
3	i think computers are scary	What do you think about machines ?
4	the bus to samarkand leaves at nine	Lets discuss further why your exams.
5	i am worried about my exams *(again)*	Does that suggest anything else which belongs to you ?
6	what is two plus two	What do you think ?
7	i am a robot made of cheese	Do you know anyone else who is a robot made of cheese ?

Investigate

1. **Keywords:** which input words seem to trigger which kind of reply (lines 2, 3, 6)?
2. **Copying:** in lines 1 and 7, which of your words came back? What changed (look at *my* → *your*)?
3. **Memory:** line 4 has no keyword at all. Where did "your exams" come from? Why is the grammar broken?
4. **Repeats:** why is reply 5 different from reply 1?
5. **Break it:** find your own input that makes ELIZA's lack of understanding obvious. Write it down.

Fallback: analyse the printed transcript above; it contains everything needed.

🎬 WATCH / EXPLORE

Symbolic AI: Crash Course AI #10 by CrashCourse with PBS Digital Studios (YouTube, 13 min, 2019)
https://www.youtube.com/watch?v=WHCo4m2VOws

Why use it?	A lively recap of everything at this stop: symbols, knowledge bases, truth tables, IF–THEN inference and expert systems, including their limits.
Which part?	**03:45–07:15** "How does it work": propositional logic, truth tables, AND/OR/NOT. **07:15–end** "Implications": if-then rules, inference, expert systems and why they are limited.
Your task	While watching 07:15–end, write down one **advantage** and one **limit** of expert systems the video gives. Compare with Section "Real expert systems" below.
Watch out	The video computes AND and OR with multiplication and addition of 1s and 0s. That is a neat trick, but in Boolean logic "1 + 1" still just means *true*. Its symbols and examples differ from ours; the ideas are the same. Captions are available.

🧭 THE REAL THING: REAL EXPERT SYSTEMS, AND THEIR LIMITS

System	When / where	What it did
DENDRAL	from 1965, Stanford	Suggested molecular structures from spectrograph data; often called the first expert system
MYCIN	from 1972, Stanford	Diagnosed blood infections with about 500 rules; explained its reasoning; about as good as specialists
XCON (R1)	Carnegie Mellon for DEC; in use from January 1980	Configured computer orders; estimated to save DEC about $25 million a year by 1986

Sources: Britannica: Expert systems; McDermott (1981); Middlesex University XCON notes.

Strengths: explainable; uses expert knowledge directly; predictable. **Limits:** no common sense: told about a patient bleeding from a gunshot wound, MYCIN would still look for a bacterial cause (Britannica). Systems know only what their rules cover (**brittle**), and big rule bases are hard to maintain because new rules can **conflict** with old ones.

2.3 Explain like the scholar (write the 'why?') ⏱ 10 min

GOAL	Produce an **explanation chain** for a conclusion, the strength that makes rule-based systems trustworthy.
PREPARATION	Your trace from Mission 2.1 round 2 (zebra).
TASK	Write BEK's answer to a traveller who asks *"Why do you say zebra?"* as an indented "because..." chain, ending at the facts the traveller gave.
OBSERVATION	Check that every line names a rule or says "you told me".
REFLECTION	Show your chain to a partner. Could they spot a wrong step from your chain alone? Why is that valuable for a doctor using MYCIN?

FEEDBACK	Model chain → Feedback Corner, Mission 2.3.

⚠️ TRAPS ON THE ROAD (COMMON MISUNDERSTANDINGS)

Trap	The truth
"The system understands animals."	It matches symbols. Rename every word to `X17` and it reasons the same way (ELIZA shows the same illusion with language).
"If the answer is right, the reasoning is right."	A trace can reach "tiger" by firing R5 too early. Check the steps, not just the answer.
"A strange answer means the engine is broken."	Usually the **knowledge base** has a gap (Tweety) or a conflict (mystery).
"Forward and backward chaining reach different conclusions."	Same rules and facts give the same conclusions; they differ in direction and in which questions they ask.

✅ CHECKPOINT (ANSWERS IN THE FEEDBACK CORNER)

1. Which part changes for every new case: the knowledge base or working memory?
2. When does forward chaining stop?
3. Rule `IF A AND B THEN C`; working memory holds only A. Does it fire?
4. In Prolog, why does `is_a(Pingu, penguin).` answer `Pingu = pingu` instead of `true`?
5. Name one thing ELIZA's conversation does **not** show, even though it looks clever.

🗝️ Feedback Corner

Try everything first. Then check here: every answer explains *why*.

Predict box. Two rules, in this order: R1 (has feathers → is a bird), then R3 (bird + cannot fly + swims → penguin).

Mission 2.1.

Round 1: penguin		
Cycle	**Fired**	**New fact**
1	R1	is a bird
2	R3	is a penguin
3	–	stop

Round 2: zebra		
Cycle	**Fired**	**New fact**
1	R2	is a mammal
2	R6	is a zebra
3	–	stop

R3 cannot fire in cycle 1 because *is a bird* is not yet on the table. **Last round:** cycle 1 R2 → mammal; cycle 2 R5 → **tiger**; cycle 3 R6 → **zebra**; cycle 4 stop. The animal is "both a tiger and a zebra"! The engine is fine: the **knowledge base** has a **rule conflict** (R5 and R6 both accept a striped mammal). A fix: add *does not have hooves* to R5, then re-test every earlier round. (Checked with `topic02_mini_rule_engine.py`.)

Mission 2.2. Goal tiger → R5 needs *is a mammal*, *eats meat*, *has black stripes*; *is a mammal* → R2 needs *gives milk*. Questions: "Does it give milk?" → "Does it eat meat?" → "Does it have black stripes?": 3 questions, stopping early at any "no". Asking about every fact would take 8 questions. A diagnosis system prefers backward chaining because it tests one suspected illness and asks only relevant questions.

Lab 2A (our SWISH results). 1: `true` (Prolog proves the goal backwards: penguin needs bird, which needs has_feathers). 2: `pingu`, `hoot`, `tweety`. 3: only `bird`: no rule covers a flying bird that does not hunt at night (a **gap**). 4: `mammal`, `tiger`, `zebra`: the **rule conflict** (Figure L2.2). 5: `Pingu = pingu`: the capital letter made `Pingu` a **variable**, so Prolog answered "which animal is a penguin?". Lower-case names matter! **Build:** `fact(gerald, gives_milk). fact(gerald, has_long_neck). fact(gerald, has_hooves).` and `is_a(X, giraffe) :- is_a(X, mammal), fact(X, has_long_neck), fact(X, has_hooves).` gives `mammal`, then `giraffe`. With the typo, Prolog simply answers **false**, because to a computer `has_long_nek` and `has_long_neck` are different symbols; it never "guesses" what you meant. **Reflect:** Prolog starts from the goal, so this is **backward chaining**.

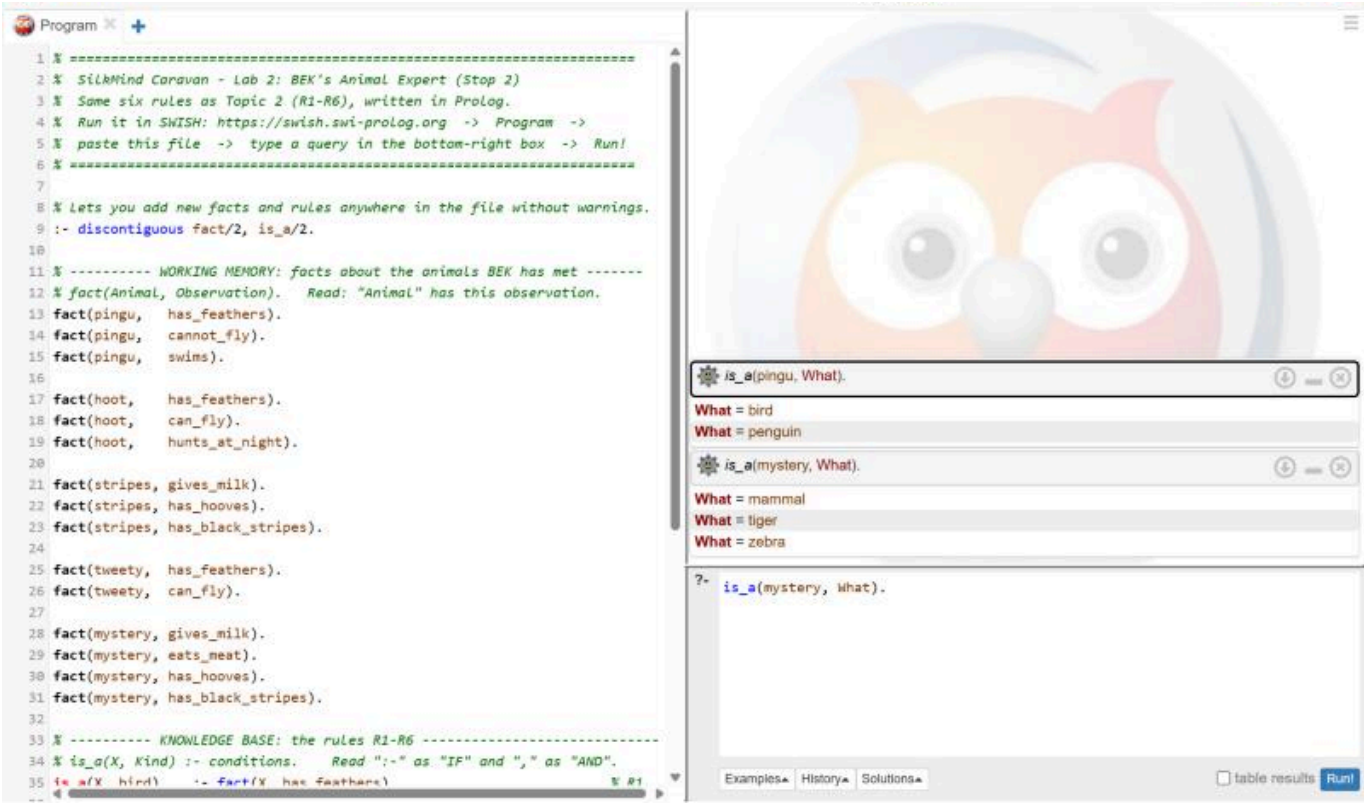


Figure L2.2 Our SWISH run of query 4: `mystery` *is a mammal, a tiger and a zebra. The rules conflict.*

Lab 2B (ELIZA). ELIZA works with a **script** of rules: it looks for a **keyword** (e.g. *mother*, *computer*, *what*, *my*, *I am*), breaks the sentence into parts with a matching **pattern**, and builds a reply from a **template**, swapping pronouns (my → your, I → you) (Weizenbaum, 1966). 1/7: your own words came back with pronouns swapped, even nonsense such as "a robot made of cheese": no understanding involved. 2/3/6: keyword families (family words → "Your mother?"; computer → machines; a question → "What do you think?"). ELIZA did **not** calculate 2 + 2. 4: no keyword, so ELIZA reused a phrase it **stored earlier** from a "my ..." sentence; pasting it into a template broke the grammar. 5: a keyword usually has several reply templates, so the same input can get a different reply. **What it does not show:** understanding, memory of facts, reasoning, or knowing what any word means. It is a powerful example of how **rules over symbols** can *look* intelligent.

Mission 2.3 (model chain). "Zebra, because R6: it is a mammal AND has hooves AND has black stripes. → It is a mammal because R2: it gives milk. → 'gives milk', 'has hooves' and 'has black stripes' are what you told me." A doctor can check each line, so a wrong step is easy to spot.

Checkpoint. 1: working memory. 2: when a whole cycle adds no new fact. 3: no; AND needs both. 4: capitalised words are variables in Prolog. 5: e.g. it does not understand meaning, cannot answer factual questions, or does not know a robot can't be made of cheese.

📮 **POSTCARD FROM THIS STOP**

Greetings from Bukhara! BEK became an **expert system**: a **knowledge base** of IF–THEN rules plus an **inference engine** that **chains** them, forwards from facts or backwards from a goal, and can **explain** every conclusion. But BEK knows only what his rules say: gaps and conflicts are his weak spots. Next stop: a parrot who has no rules at all...

STOP 3 · SAMARKAND

To'ti's Market



What if nobody writes the rules, and the machine works them out from examples?

Markdown source: `03-symbolic-ai-vs-machine-learning.md` · Outcome LO5 · ≈ 65 min · Missions 3.1–3.2 · Lab 3 (Machine Learning for Kids)

🐪 CARAVAN TALE (STORY, NOT A TECHNICAL CLAIM)

Samarkand's Siyob Bazaar is loud and busy, and so is the merchants' message board. Mixed in with real trade offers are scams. BEK writes rules: `IF message contains "FREE MONEY" THEN scam`. By lunchtime the scammers are writing "FR3E M0NEY" and BEK's rule catches nothing.

To'ti the parrot has watched the merchants all morning. Nobody gave him rules, but he has seen hundreds of messages and heard which ones the merchants called "scam". He starts squawking at suspicious messages, and he is mostly right.



TO'TI THE PARROT

I don't know any rules. I just noticed that scams tend to have lots of links. Seen it before!



NAVIGATOR NODIRA

To'ti is doing **machine learning**: instead of a person writing the rule, the rule is **chosen from labelled examples**. Let's do exactly what he did, by hand.

🔮 PREDICT FIRST

You will see ten messages, each with its number of links and a label (scam / not). To'ti may learn one rule: `IF links ≥ k THEN scam`. Guess now: which **k** will make the fewest mistakes? Will the best rule be perfect?

3.1 Teach To'ti: learn a rule from data

⏱ 20 min

GOAL	See what **training** means: try possible rules, count mistakes on **labelled data**, keep the best one. Then see how new data changes the **model**.
PREPARATION	Message cards A–J and N (Appendix A), a pencil as the "threshold marker", or just the table below.

TASK

Message	A	B	C	D	E	F	G	H	I	J
Links	0	1	1	2	3	2	4	5	6	8
Label	not	not	not	not	not	**scam**	**scam**	**scam**	**scam**	**scam**

1. Lay the cards in a line, sorted by number of links.
2. Place the pencil just before the cards with **k = 2** links: every card from the pencil onwards is predicted "scam". Count the mistakes (wrong in *either* direction).
3. Repeat for every k from 0 to 9. Choose the k with the fewest mistakes: that is your **model**.
4. **Predict** three new messages: K (7 links), L (0 links), M (3 links).
5. **Retrain:** a new labelled card arrives: **N, 3 links, scam**. Add it and recount for k = 2, 3, 4 and 5.

OBSERVATION

k	0	1	2	3	4	5	6	7	8	9
Mistakes (10 cards)										
Mistakes (11 cards, with N)	–	–					–	–	–	–

REFLECTION

Was your Predict-box guess right? Is the best rule perfect? Who "wrote" the number k? What happened after card N arrived? A classmate says: *"It made only 1 mistake in 10, so it will be 90% right on every future message."* What is wrong with that claim?

FEEDBACK

Hint: check message F carefully. Full table and explanation → Feedback Corner, Mission 3.1. Optional check: `supporting-files/examples/topic03_learn_a_rule.py`.

🧭 THE REAL THING: MACHINE LEARNING IN A NUTSHELL

Machine learning (ML) is a branch of AI in which software learns patterns from data, instead of following rules a programmer wrote in advance (Britannica: Machine learning; IBM Developer).

Term	Meaning	In Mission 3.1
Labelled data	Examples that come with the correct answer	10 cards marked scam / not
Feature	A measurable property the program may use	number of links
Training	Searching for the pattern that best fits the labelled data	counting mistakes for each k
Model	The result of training: a learned "rule"	`IF links ≥ 4 THEN scam`
Prediction	The model's answer for a new example	K → scam

Real models use thousands of features and millions of examples, and their learned "rule" is stored as huge numbers of values. That is why they are flexible but hard to explain, even though the core idea is the one you just did by hand.

Figure 3.1 Two ways to give a computer knowledge

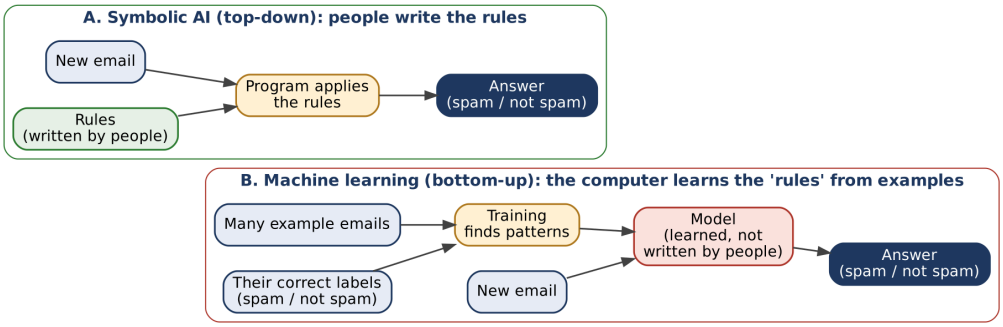


Figure 3.1 Two ways to give a computer knowledge: people write the rules (BEK), or training finds them in labelled examples (To'ti).

🧪 LAB: 3 · TRAIN A SCAM DETECTOR (MACHINE LEARNING FOR KIDS)

Goal. Train a real text classifier, test it on messages it has never seen, investigate its mistakes, and improve the training data. **Platform:** Machine Learning for Kids by Dale Lane: machinelearningforkids.co.uk. **Access:** free; choose **Get started → Try it now** to work without an account. A Cloudflare **"Verify you are human"** box appears first: tick it yourself. "Try it now" projects are temporary, so finish in one session. Button names below follow the site's own worksheets; small wording differences are possible.

Steps

1. **+ Add a new project** → name `Caravan messages` → type **recognising text** → **Create**. Open the project → **Train**.
2. **+ Add new label** twice: `genuine` and `scam`.
3. Use **+ Add example** to type these 16 training messages into the right labels:

genuine

1. Your order of twenty silk scarves will arrive on Tuesday
2. Can we meet at the bazaar at noon to agree the price
3. The camel caravan is delayed by one day because of a sandstorm
4. Thank you for your payment, here is your receipt
5. Please bring the spice samples to my shop tomorrow
6. We have new carpets from Bukhara, visit us this week
7. Your room at the caravanserai is booked for three nights
8. The market closes early on Friday for the festival

scam

1. Congratulations you have won a free camel, click this link now
2. URGENT your account is locked, send your password today
3. You are selected for a secret gold prize, pay a small fee to claim
4. Cheap silk 90 percent off only today, send money first
5. Final warning: your parcel is held, pay the release fee now
6. Click here to claim free gold coins, limited time
7. Your bank details are needed to receive your inheritance
8. Win win win, act now, free prize waiting

4. **< Back to project → Learn & Test → Train new machine learning model**. Wait until training finishes (it can take a few minutes).
5. Type each **test message** into the test box, one at a time. Record the label and how confident the model says it is.

#	Test message (never seen in training)	Your prediction	Model says	Right?
T1	Free tea tasting at our shop on Friday			
T2	Your parcel from Samarkand will arrive tomorrow			
T3	Send your password to unlock your prize			
T4	Please confirm the meeting time for the silk delivery			
T5	CONGRATULATIONS			
T6	The weather in Kashgar is sunny			

Investigate

1. Which test messages did the model get wrong? For each, find the **words** it shares with the training examples of the other label.
2. Improve the data: add **three** genuine messages that contain "free", "parcel" or "prize" in an honest way (e.g. *"Free delivery for orders over ten carpets"*). Retrain and retest T1 and T2.
3. T6 is about the weather: neither genuine trade nor scam. What does the model do with a message unlike anything it has seen?
4. Could BEK have written rules for this? Which would be easier to keep up to date when scammers change their words?

Fallback (no access, blocked site, or the human check fails): open the notebook `SilkMind_Labs.ipynb`, **Part 3**. It trains a tiny word-counting scam detector on exactly these 16 messages and tests T1–T6, so you can answer every Investigate question offline. It is much simpler than Machine Learning for Kids, so its answers can differ; the Feedback Corner explains both.

🎬 WATCH / EXPLORE

How AI Actually Learns (first published as "How Machines Learn") by CGP Grey (YouTube, 8 min 54 s, December 2017)
https://www.youtube.com/watch?v=R9OHn5ZF4Uo

Why use it?	A funny, animated explanation of why people stopped writing every rule by hand for big problems, and how "teacher bots" test "student bots" against an answer key.
Which part?	Watch from the start to about **7:20**. From 0:16: why hand-written if-this-then-that bots don't scale. The middle section: builder bots, teacher bots and tests. **5:12**: why more data means better tests.
Your task	Compare: the teacher bot's answer key ↔ our labels; keeping the best student ↔ keeping the k with fewest mistakes. Write one similarity and one difference.
Watch out	The builder/teacher-bot story is a **simplified analogy**. Grey's own follow-up video, *How AI, Like ChatGPT, *Really* Learns*, explains that most real systems are trained by gradually adjusting numbers (gradient descent), not by breeding bots. No YouTube? Mission 3.1 covers the same idea.

3.2 BEK or To'ti? Choose the right tool

⏱ 10 min

GOAL	Compare symbolic AI and ML and **justify** a choice using clear criteria.
PREPARATION	The comparison table below.
TASK	For each job, decide: **BEK** (rules), **To'ti** (learning) or **both**. Then argue it out with a partner who must disagree. 1. Checking whether a student meets a course's published entry grades. 2. Recognising which animal is in a photo. 3. Calculating a delivery fee from a published price list. 4. Recommending songs from what millions of users have played. 5. A hospital system that reads X-ray images **and** must flag any result breaking a fixed safety guideline.
OBSERVATION	Record your choice and a reason for each, using a criterion from the table.
REFLECTION	Which criterion decided most of your choices?
FEEDBACK	Answers → Feedback Corner, Mission 3.2.

Question	BEK: symbolic AI	To'ti: machine learning
Where does knowledge come from?	People write rules	Patterns in labelled data
Needs to start	Experts who can state rules	Enough good, labelled data
Can it explain an answer?	**Yes**: it lists the rules that fired	Often difficult at real-world scale

Unusual case	Fails if no rule covers it (brittle)	May cope if similar examples existed; may be confidently wrong
Updating	Edit rules by hand	Collect data and retrain

⚠️ TRAPS ON THE ROAD (COMMON MISUNDERSTANDINGS)

Trap	The truth
"ML means the computer understands."	In Mission 3.1 the program never knew what a link or a scam is. It reduced mistakes.
"A model trained on real data is always right."	Our best model still gets F wrong, and a model is only as good as its data.
"ML doesn't use rules."	A model *is* a kind of rule (ours literally is). The difference is **who writes it**.
"Symbolic AI has been replaced."	Rules are still used wherever exact, explainable decisions are required (Stop 5).

✅ CHECKPOINT (ANSWERS IN THE FEEDBACK CORNER)

1. What is a **label**? Give an example from Mission 3.1.
2. How many training mistakes does `IF links ≥ 5 THEN scam` make, and which messages?
3. Why did adding message N change the best rule?
4. A bank must explain every loan refusal using its published criteria. BEK or To'ti? Why?

🗝️ Feedback Corner

Try everything first. Then check here: every answer explains *why*.

Mission 3.1. Mistakes for k = 0…9: **5, 4, 2, 2, 1, 2, 3, 4, 4, 5**. Best: **k = 4** with 1 mistake (message F: 2 links but scam). So the model is `IF links ≥ 4 THEN scam`. It is **not** perfect, and nobody wrote the number 4: the data chose it. Predictions: K → scam, L → not, M → not. With N (3 links, scam) added: k = 2 → 2 mistakes (D, E); k = 3 → 2 (E, F); k = 4 → 2 (F, N); k = 5 → 3 (F, G, N). Now k = 2, 3 and 4 **tie**: one new example changed the model, and training needs a tie-break rule (our program picks the smallest k). The "90% on every future message" claim is wrong: the 1-in-10 was measured on the **same** messages used to choose k, and 10 examples are far too few. Real accuracy must be measured on **new labelled messages not used in training**. (All numbers checked with `topic03_learn_a_rule.py`.)

Lab 3 (expected outcomes). Every training run starts fresh, so your exact results and confidence values can differ from a classmate's. The model learns which **words** go with which label. Watch especially **T1** ("Free tea tasting…") and **T2** ("Your parcel…"): they are genuine but share words (*free*, *parcel*) with the scam examples, so the model may call them scam. **T3** and **T5** share many scam words and are likely to be labelled scam. **T6** is unlike every example; the model must still pick one of its two labels. It cannot say "neither", which shows that a model only knows the categories it was trained on. Adding honest examples containing "free"/"parcel"/"prize" and retraining usually moves T1/T2 towards *genuine*: better data, better model. BEK's word rules would need constant hand-editing as scammers change spelling; retraining on new labelled messages is easier to keep up to date, but harder to explain.

Offline version (notebook Part 3, our run): T1 genuine, T2 genuine, T3 scam, T4 genuine, T5 scam, **T6 genuine**. T6 is called genuine only because it contains *the* and *is*, words that appeared more often in genuine training messages. The model learned "the = genuine", an accident of a small data set. A model learns whatever patterns are in its data, sensible or not.

Mission 3.2. 1 BEK: explicit published rules; must be explainable. 2 To'ti: no one can write rules for every photo. 3 BEK: exact price list. 4 To'ti: patterns too complex to write; plenty of data. 5 Both: ML reads images; the fixed guideline is a rule that must be checked and explained. Accept other answers that use a criterion from the table.

Checkpoint. 1: the correct answer attached to an example, e.g. message F → "scam". 2: **2** (F and G: both scam with fewer than 5 links). 3: the model depends on the data; N made several thresholds equally good. 4: **BEK**: published criteria and a required explanation.

📮 **POSTCARD FROM THIS STOP**

Greetings from Samarkand! To'ti showed that a computer can **learn** a rule: try options, count mistakes on **labelled data**, keep the best. The result is a **model**: flexible, but imperfect, data-hungry and harder to explain. BEK writes rules; To'ti learns them. Next stop: the ruins where people stopped believing in BEK...

STOP 4 · OTRAR

The Frozen Caravanserai

Why did the world stop believing in rule-based AI, twice?

Markdown source: `04-history-and-ai-winters.md` · Outcome LO6 · ≈ 55 min · Missions 4.1–4.4

🐪 CARAVAN TALE (STORY, NOT A TECHNICAL CLAIM)

The caravan camps beside the ruins of Otrar, once a great trading city on the Syr Darya river, now silent mounds of earth. Around the fire, Nodira tells a story about trading towns that boom when everyone believes in them, then empty when the promises fail. *"The same thing happened to BEK's kind,"* she says. *"Twice."*

(The city is real; the comparison between a deserted trading town and an "AI winter" is our analogy, not history.)

BEK THE RULE-ROBOT
Twice? But rules work! You saw me identify penguins!

NAVIGATOR NODIRA
You identify penguins well, BEK. The trouble came when people promised you could do *everything*. Let's rebuild the story ourselves.

🧭 THE REAL THING: WHAT IS AN AI WINTER?

An **AI winter** is a period in which funding for and interest in AI research fall sharply, usually after expectations were set too high and results did not match (Wikipedia: AI winter). Symbolic AI went through the same cycle twice:

big promises → funding → results fall short → funding cut ("winter")

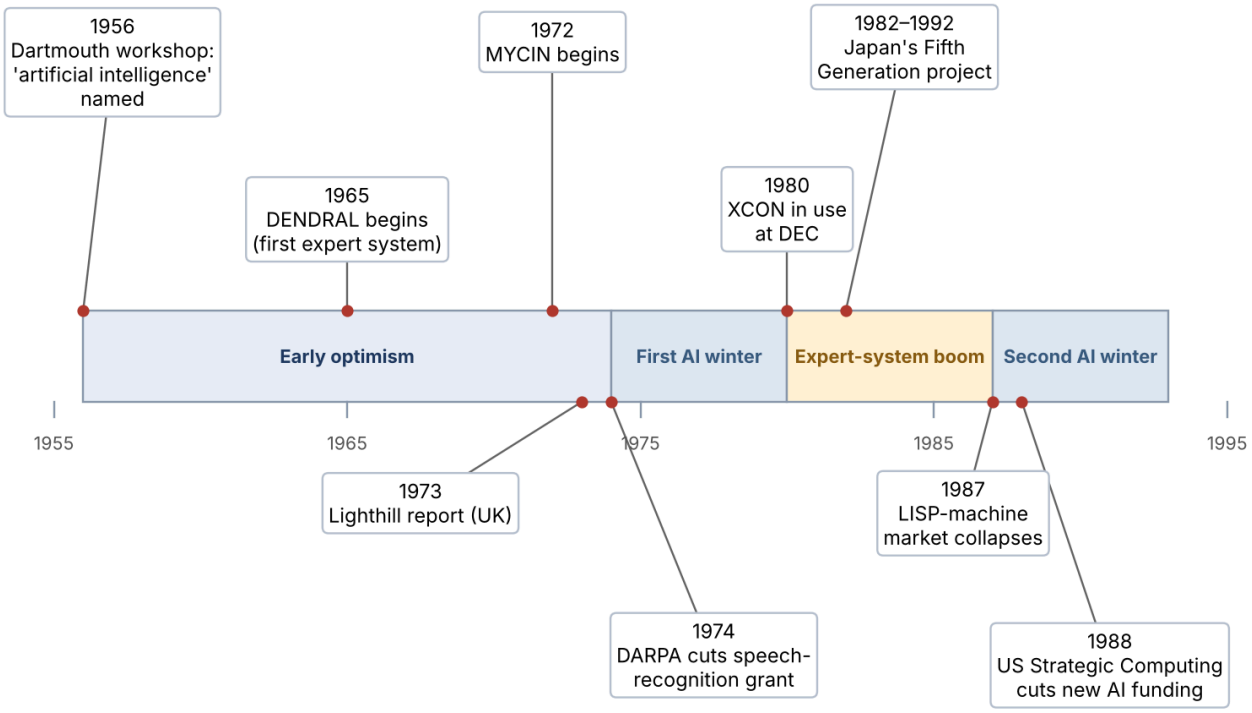
4.1 Timeline cards: boom or winter?

⏱ 12 min

GOAL	Place the key events of 1956–1993 in order and spot the two cycles of promise and disappointment.
PREPARATION	Event cards 1–9 (Appendix A), a long strip of paper as a timeline.
TASK	1. Without looking at Figure 4.1, put the nine event cards in time order. 2. Sort each card into **Optimism / Boom** (green) or **Winter / Disappointment** (blue) and mark your timeline. 3. Now compare with Figure 4.1 and fix any mistakes.
OBSERVATION	How many events did you place correctly the first time? Where on your strip do the two "winters" begin?
REFLECTION	Name one thing that happened just **before** each winter started. Is there a pattern?

FEEDBACK	Answers → Feedback Corner, Mission 4.1.

Figure 4.1 Symbolic AI, 1956–1993: two cycles of promise and disappointment



Start and end years of the AI winters are approximate; historians give slightly different ranges.

Figure 4.1 Symbolic AI 1956–1993: two cycles of promise and disappointment. Winter start and end years are approximate.

4.2 Crack the lock: meet the combinatorial explosion

⏱ 10 min

GOAL

Feel why some problems become impossible to solve by checking every possibility, a key reason for the first AI winter.

PREPARATION

Calculator or phone. There are 86,400 seconds in a day and about 31.5 million in a year.

TASK

A thief tries every code on a lock with digits 0–9 at **one guess per second**.

Digits	Possible codes	Time to try them all
1	10	10 seconds
2	100	
4		
6		
8		

Then: a computer a **million** times faster tries a **16-digit** code. How long now?

OBSERVATION

What happens to the number of codes each time you add one digit? Did the million-times-faster computer "solve" the problem?

REFLECTION	Early AI programs searched through possibilities (moves, plans, sentences). Why did programs that worked on small "toy" problems fail on real ones?
FEEDBACK	Answers → Feedback Corner, Mission 4.2.

🧭 THE REAL THING: WHY THE WINTERS HAPPENED

First winter (≈1974–1980). Early researchers promised human-like intelligence within a generation. Programs worked on small toy problems but failed on real ones such as understanding ordinary language. In **1973** Sir James Lighthill's review for the UK Science Research Council concluded that *"in no part of the field have the discoveries made so far produced the major impact that was then promised"* and pointed to the **combinatorial explosion** (Mission 4.2). UK funding was sharply reduced, and in the USA DARPA cut grants such as a large speech-recognition project (1974).

The boom (1980s). Expert systems such as XCON (in use from 1980) made money, and companies built many more. Japan's **Fifth Generation** project (1982–1992) aimed at computers built for knowledge-based reasoning.

Second winter (≈1987–1993). Expert systems ran into exactly what you saw at Stop 2: they were **brittle** (gaps, like Tweety), **costly to maintain** (conflicts, like the mystery animal) and could not **learn**. The market for special LISP-machine computers collapsed in 1987 as cheaper ordinary computers caught up, and new US Strategic Computing funding for AI stopped in 1988. The Fifth Generation project fell well short of its goals.

Afterwards: rules did not disappear: they became an ordinary software tool (Stop 5), while research moved toward learning from data (Stop 3).

Sources: Lighthill (1973); Wikipedia: AI winter; IPSJ Computer Museum; Nature 356 (1992); McDermott (1981).

4.3 The fact-checker's desk

⏱ 12 min

GOAL	Check a historical claim against more than one source, and decide what to do when sources disagree.
PREPARATION	Internet access. The two sources below.
TASK	A website says: *"Japan's Fifth Generation project ran from 1981 to 1991 and cost $850 million."* 1. Open the **IPSJ Computer Museum** page: museum.ipsj.or.jp/en/computer/other/0002.html. Find the start year, end year and cost. 2. Open the **Nature** news report of 26 March 1992: nature.com/articles/356273b0.pdf. What does it say about the dates, spending and results?
OBSERVATION	Fill a table: *claim part · IPSJ says · Nature says · confirmed?* for dates, cost and outcome.
REFLECTION	When two good sources give different numbers, what should a careful writer do?

FEEDBACK	No internet? The key facts are in the Feedback Corner. Answers → Feedback Corner, Mission 4.3.

4.4 It's 1985: pitch an expert system

⏱ 12 min

GOAL	Use Stop 2 evidence to **predict** why an over-ambitious expert-system project would struggle.
PREPARATION	Stop 2 Feedback Corner (Tweety gap, mystery conflict). Groups of 3: one **Pitcher**, two **Critics**.
TASK	The Pitcher has 1 minute to sell: *"An expert system that answers **any** customer question about **any** of our 5,000 products!"* The Critics have 2 minutes to predict problems, each backed by something you **saw** at Stop 2. Then swap roles once.
OBSERVATION	List the Critics' predicted problems and the Stop 2 evidence for each.
REFLECTION	Which problem would get **worse** as the product list grows? Connect it to Mission 4.2.
FEEDBACK	Answers → Feedback Corner, Mission 4.4.

📜 EXPLORE A PRIMARY SOURCE

Artificial Intelligence: A General Survey, Sir James Lighthill, Science Research Council (UK), 1973. Copy hosted by the University of Edinburgh: aiai.ed.ac.uk/events/lighthill1973/lighthill.pdf

Why use it?	This is the actual report that helped start the first AI winter, not someone's summary of it.
Which part?	Use your PDF viewer's search (Ctrl+F) for **"major impact"** and **"combinatorial explosion"**; read the paragraph around each.
Your task	In one sentence each: what did Lighthill say had gone wrong, and what is the combinatorial explosion in his words?
Watch out	It is a 1970s academic report: long and formal. Read only those two passages. Lighthill's view was also strongly disputed by AI researchers at the time.

⚠️ TRAPS ON THE ROAD (COMMON MISUNDERSTANDINGS)

Trap	The truth
"In an AI winter, all AI research stopped."	Funding and interest fell sharply, but research continued on a smaller scale; some systems (XCON) stayed in use.
"Symbolic AI failed, so it was abandoned."	It failed to deliver *general* intelligence, but rule-based methods are still everywhere (Stop 5).
"The winters were only about slow computers."	Over-promising mattered as much: both winters followed promises far ahead of results.
"History dates are fixed facts."	Sources disagree (Mission 4.3). Prefer original records, give ranges, and cite.

✅ CHECKPOINT (ANSWERS IN THE FEEDBACK CORNER)

1. Put in order: XCON enters use · Lighthill report · Dartmouth workshop · LISP-machine market collapses.
2. Give one **expectations** cause and one **technology** cause of the second AI winter.
3. True or false: "During the AI winters, all expert systems were switched off." Explain.
4. How does Stop 2's "mystery animal" help explain why big expert systems were costly?

🗝️ Feedback Corner

Try everything first. Then check here: every answer explains *why*.

Mission 4.1. 1956 Dartmouth workshop ("artificial intelligence" named) → 1965 DENDRAL begins → 1972 MYCIN begins → 1973 Lighthill report → 1974 DARPA cuts → 1980 XCON in use → 1982 Fifth Generation project starts → 1987 LISP-machine collapse → 1988 Strategic Computing cuts. Boom/optimism: 1956, 1965, 1972, 1980, 1982. Winter: 1973–74 and 1987–88. Pattern: each winter followed a period of big promises (general AI soon; the 1980s expert-system and Fifth Generation hopes).

Mission 4.2. 2 digits: 100 codes, 100 s. 4: 10,000 codes ≈ 2.8 hours. 6: 1,000,000 codes ≈ 11.6 days. 8: 100,000,000 codes ≈ 3.2 years. Each extra digit multiplies the work by 10: that is the **combinatorial explosion**. 16 digits at a million guesses per second: $10^{16} \div 10^{6} = 10^{10}$ seconds ≈ **317 years**. A faster computer does not solve the problem, it only moves the wall. Toy problems had few possibilities; real problems have explosively many.

Mission 4.3. IPSJ: project **1982–1992** (11 years), led by ICOT under MITI, about **¥54 billion**; built several parallel inference machines. *Nature* (March 1992): launched 1982, a ten-year project extended by a year; "over $400 million" spent; far short of its goals. So the **dates** in the claim are wrong (1981–1991) and the **$850 million** figure is not supported by either source; the claim that it fell short **is** confirmed. A careful writer prefers the project's own records and reports from the time, gives a range or says "about", and cites the source. (Our own Markdown V1.1 had the wrong dates; we corrected them this way.)

Mission 4.4. Good predictions: **gaps**: customers ask things no rule covers, as with Tweety; **conflicts and maintenance**: thousands of rules, every new product needs new rules that may clash, like R5/R6; **expert time**: experts must state every rule and may disagree; **language**: understanding free-form questions

defeated early AI. Conflicts and maintenance get worse as the list grows: the number of rule combinations that could clash grows explosively (Mission 4.2).

Checkpoint. 1: Dartmouth (1956) → Lighthill (1973) → XCON (1980) → LISP-machine collapse (1987). 2: expectations, e.g. expert systems and the Fifth Generation project promised far more than they delivered; technology, e.g. brittle, costly to maintain, could not learn, or LISP machines overtaken by cheaper computers. 3: **False**: funding fell, but research continued and some systems stayed in use. 4: rules can conflict (tiger *and* zebra); finding and fixing conflicts among thousands of rules is expensive.

📮 **POSTCARD FROM THIS STOP**

Greetings from Otrar! Rule-based AI went through two **AI winters** (≈1974–80, ≈1987–93). The causes were promises far ahead of results, the **combinatorial explosion**, and expert systems that were **brittle** and costly to maintain. But BEK's kind survived... Next stop: where rules live today.

STOP 5 · KASHGAR

The Modern Bazaar



Where are BEK's rules hiding in the apps you use every day?

Markdown source: `05-symbolic-ai-in-modern-software.md` · Outcome LO7 · ≈ 60 min · Missions 5.1–5.3 · Lab 5 (Prolog knowledge graph)

🐫 CARAVAN TALE (STORY, NOT A TECHNICAL CLAIM)

Kashgar's great bazaar has traded for centuries; today half the stalls take payment by phone. BEK expected to find himself in a museum. Instead he finds his rules everywhere: in the payment app, in the game the stall-keeper's son is playing, in the delivery app's fee calculator.



BEK THE RULE-ROBOT

They don't call me "AI" any more. They call me "business logic". I'm not sure if that's a promotion.



TO'TI THE PARROT

And look: some apps use *both* of us. I guess the delivery time, then a rule adds 10 minutes when it rains!

🔮 PREDICT FIRST

Name **three** decisions an app on your phone makes that you think are simple IF–THEN rules (not learning). Keep your list for Mission 5.1.

5.1 Rule hunt: forms and apps

⏱ 10 min

GOAL	Recognise rule-based decisions in everyday software and write them as IF–THEN rules.
PREPARATION	Any website with a sign-up or contact form; one app you use often.
TASK	1. Open a web form and press **Submit** while a required box is empty. Write down exactly what happens. 2. Write that behaviour as one IF–THEN rule. 3. In your app, find **two** more rule-based decisions and **one** feature that probably uses machine learning.
OBSERVATION	Table: *feature · rule or learning? · your evidence*.

REFLECTION	What made you decide that a feature was "learning" and not "rules"? Could any feature be both?
FEEDBACK	Hint: browsers check a field marked `required` before a form is sent (MDN Web Docs). Answers → Feedback Corner, Mission 5.1.

🧭 THE REAL THING: RULES INSIDE MODERN SOFTWARE

Forms. When a required field is empty, the browser blocks the form and shows a message such as "Please fill out this field" (MDN: Client-side form validation): `IF field is required AND empty THEN block AND show message`.

Business rules engines keep an organisation's decision rules separate from the program so staff can change them. Commercial products such as Salesforce's Business Rules Engine organise them in **decision tables** and use them for eligibility, discounts and quotes (Salesforce Trailhead). A **decision table** has one rule per row: conditions on the left, result on the right. A good table covers every case **exactly once**.

Game characters often use a **finite state machine**: the character is in one **state** (Wander, Attack, Flee) and a **transition rule** moves it to another; game-design teaching material calls this the most common pattern for game AI (Principles of Computer Game Design, Lecture 23).

Rule	IF current state is…	AND…	THEN new state
G1	Wander	player is seen	Attack
G2	Attack	health is low	Flee
G3	Attack	no player is present	Wander
G4	Flee	no player is present	Wander

If no rule matches, the character **stays in its current state**.

5.2 Guard duty (role-play a finite state machine) ⏱ 12 min

GOAL	Trace a rule-based game character through events, and discover behaviour the rules **don't** cover.
PREPARATION	Rules G1–G4 above. Two players: the **Guard** (holds a card showing the current state) and the **Game** (announces events).
TASK	The Guard starts in **Wander** with full health and may only change state using G1–G4. The Game announces five moments: 1. Player seen, health full 2. Player not present, health full 3. Player seen, health low 4. Player seen, health low 5. Player not present, health low Before playing, **predict** the Guard's state after each moment. Then play it.

OBSERVATION	Record: moment · state before · rule used · state after. Compare with your prediction.
REFLECTION	At which moment did your prediction go wrong (if any)? Write the missing rule a designer would need so a fleeing guard with recovered health attacks again.
FEEDBACK	Hint: in each moment only rules for the **current** state can apply. Answers → Feedback Corner, Mission 5.2.

5.3 The delivery-fee bug (find the error in a decision table)

⏱ 8 min

GOAL

Check a decision table for gaps and overlaps, the same boundary skill as Stop 1's 18 °C.

PREPARATION

Stop 1, Mission 1.3 Task B.

TASK

Rule	Distance (km)	Fee
F1	less than 5	€2
F2	more than 5 and at most 10	€4
F3	more than 10	€6

A customer exactly 5 km away gets an error. Test the distances 4, 5, 5.5, 10 and 11 km against each rule, find the problem and fix the table.

OBSERVATION

For each test distance write which rules match. Every distance must match **exactly one** rule.

REFLECTION

What would go wrong if you fixed the gap by changing F1 to "at most 5" *and* F2 to "at least 5"?

FEEDBACK

Answers → Feedback Corner, Mission 5.3.

🧭 THE REAL THING: KNOWLEDGE GRAPHS AND NEURO-SYMBOLIC AI

A **knowledge graph** stores facts as **things** connected by named **relationships** (*Registan → located in → Samarkand*). Google's Knowledge Graph, introduced in 2012 to help search understand "things, not strings", started with over 500 million objects and 3.5 billion facts (Google, 2012). With one rule, a system can **derive** facts nobody stored:

```
L1: IF X is located in Y AND Y is located in Z THEN X is located in Z
```

Neuro-symbolic AI combines a learned model with symbolic reasoning. In **AlphaGeometry** (Google DeepMind, 2024) a symbolic deduction engine proves geometry facts step by step, like forward chaining; when it gets stuck, a language model suggests one useful new construction (a point, line or circle). It solved 25 of 30 olympiad geometry problems in the time limit; the previous best automatic method solved 10, and the average human gold medallist 25.9 (DeepMind, 2024). BEK provides checkable proof; To'ti provides intuition.

🧪 LAB: 5 · THE SILK ROAD KNOWLEDGE GRAPH (SWISH PROLOG)

Goal. Store a tiny knowledge graph in Prolog, let a rule derive facts nobody typed in, extend the graph, and investigate a rule that never stops. **Platform:** SWISH, as in Lab 2A. **File:** `supporting-files/lab-files/lab5_silk_road_graph.pl` . Tested 3 Nov 2026.

```
located_in(registan,   samarkand).
located_in(samarkand,  uzbekistan).
located_in(uzbekistan, central_asia).

inside(X, Y) :- located_in(X, Y).                  % directly located in
inside(X, Z) :- located_in(X, Y), inside(Y, Z).    % rule L1: follow the links
```

Investigate (predict, then run)

#	Query	Prediction	Result
1	`located_in(registan, central_asia).`		
2	`inside(registan, central_asia).`		
3	`inside(registan, Where).` (press Next)		

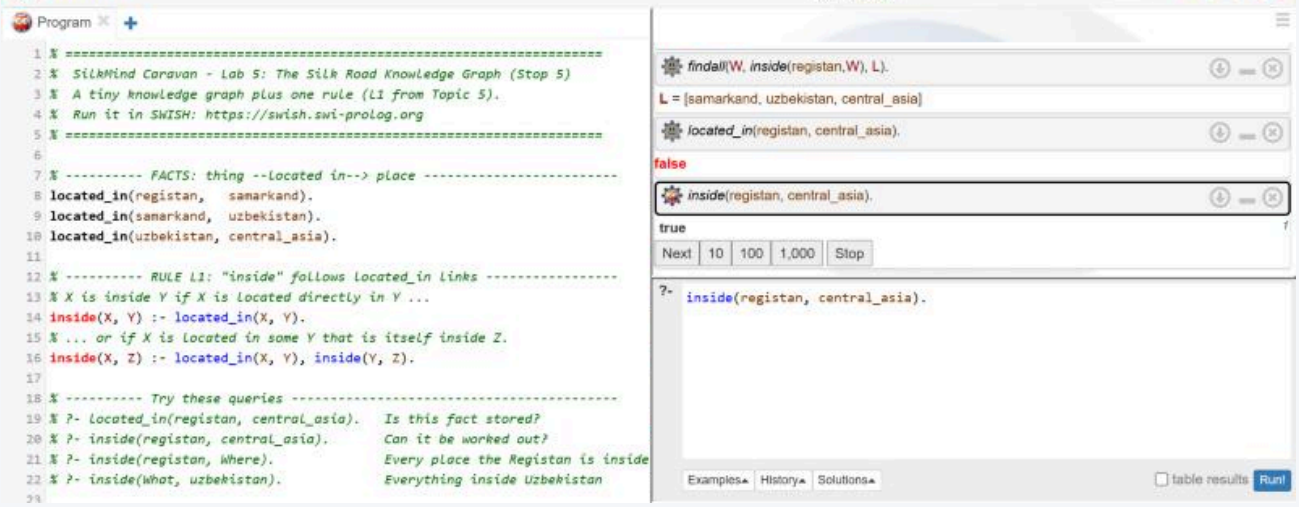


*Figure L5.1 Our SWISH run. The fact is **not** stored (`false`), yet the rule **derives** it (`true`). `findall` collects every answer into one list.*

Extend: add `located_in(ark_of_bukhara, bukhara).` and `located_in(bukhara, uzbekistan).` **next to the other facts**, then ask `inside(ark_of_bukhara, central_asia).` and `inside(What, central_asia).`

Break it (on purpose): change the second rule to `inside(X, Z) :- inside(X, Y), inside(Y, Z).` and ask `inside(registan, mongolia).` Read the error message. What do you think Prolog was doing?

Fallback: Figures L5.1 and L5.2 show our runs; trace rule L1 on paper using the table method from Stop 2.

🎬 **WATCH / EXPLORE**

AlphaGeometry: An Olympiad-level AI system for geometry, Google DeepMind blog, 17 January 2024: deepmind.google/blog/alphageometry-an-olympiad-level-ai-system-for-geometry/

Why use it?	A real, recent system where BEK-style rules and To'ti-style learning work as a team.
Which part?	The opening section and the diagram/animation showing the loop between the language model and the symbolic engine.
Your task	Draw the loop as a cycle with two boxes. Label which box is "symbolic" and which is "learned", and what each passes to the other.
Watch out	It's written by the company that built the system, so it emphasises successes. The geometry itself is advanced; you only need the loop.

⚠️ **TRAPS ON THE ROAD (COMMON MISUNDERSTANDINGS)**

Trap	The truth
"If it isn't machine learning, it isn't AI."	Rule-based reasoning is the original AI; it's just rarely called that today.
"A clever game character must be learning."	Most follow hand-written state rules. Mission 5.2's moment 3 shows behaviour coming only from the rules.
"A knowledge graph only knows what was typed in."	Rules derive new facts from links (Lab 5, query 2).
"Neuro-symbolic means learning replaced rules."	Both stay: in AlphaGeometry the symbolic engine does the proof.

✅ **CHECKPOINT (ANSWERS IN THE FEEDBACK CORNER)**

1. A guard in **Attack** sees the player and has full health for three moments. Which state is it in afterwards, and why?
2. A cinema charges: child *under 12* €5, adult *over 12 and under 65* €10, senior *65 or over* €7. Which age is not covered?
3. Facts: Bishkek → Kyrgyzstan, Kyrgyzstan → Central Asia. What can rule L1 derive?
4. In AlphaGeometry, which part is symbolic AI?

🗝️ Feedback Corner

Try everything first. Then check here: every answer explains *why*.

Mission 5.1. The browser blocks the form and shows a message (e.g. "Please fill out this field"): `IF required field is empty THEN block submission AND show message`. Typical rules: minimum order value disables Checkout; closed shop hides "Order"; password too short shows an error. Typical ML: "recommended for you", face unlock, spam filtering, voice typing. Evidence for learning: it adapts to you

or to millions of users and no fixed rule could list all cases. Many features are **both** (a learned estimate plus a fixed rule).

Mission 5.2.

Moment	State before	Rule	State after
1	Wander	G1	Attack
2	Attack	G3	Wander
3	Wander	G1 (health doesn't matter in Wander)	**Attack**
4	Attack	G2	Flee
5	Flee	G4	Wander

The usual wrong prediction is moment 3 ("it should flee"); but G2 needs state *Attack*, so the guard attacks first and flees at moment 4. Missing rule: `G5: IF state is Flee AND health is not low AND player is seen THEN Attack`.

Mission 5.3. 4 km → F1; **5 km → no rule** (not "less than 5", not "more than 5"): a **gap**; 5.5 → F2; 10 → F2; 11 → F3. Fix: F2 = **at least 5** and at most 10. Changing F1 to "at most 5" *and* F2 to "at least 5" creates an **overlap**: 5 km matches two rules with different fees.

Lab 5 (our SWISH results). 1: **false**: that fact is not stored. 2: **true**: derived with rule L1 (Registan → Samarkand → Uzbekistan → Central Asia). 3: `samarkand`, `uzbekistan`, `central_asia`. Extend: `inside(ark_of_bukhara, central_asia)` is true, and `inside(What, central_asia)` lists uzbekistan, registan, samarkand, ark_of_bukhara, bukhara. Break it: SWISH reports **"Stack limit exceeded ... Probable infinite recursion"** (Figure L5.2). The broken rule's first step is `inside(X, Y)`, the very thing it is trying to prove, so Prolog keeps calling the same rule without ever reaching a fact: it never stops. Our working rule always takes one real step (`located_in`) first. Lesson: rule order and wording matter, and a rule base can fail in ways that have nothing to do with "knowledge".

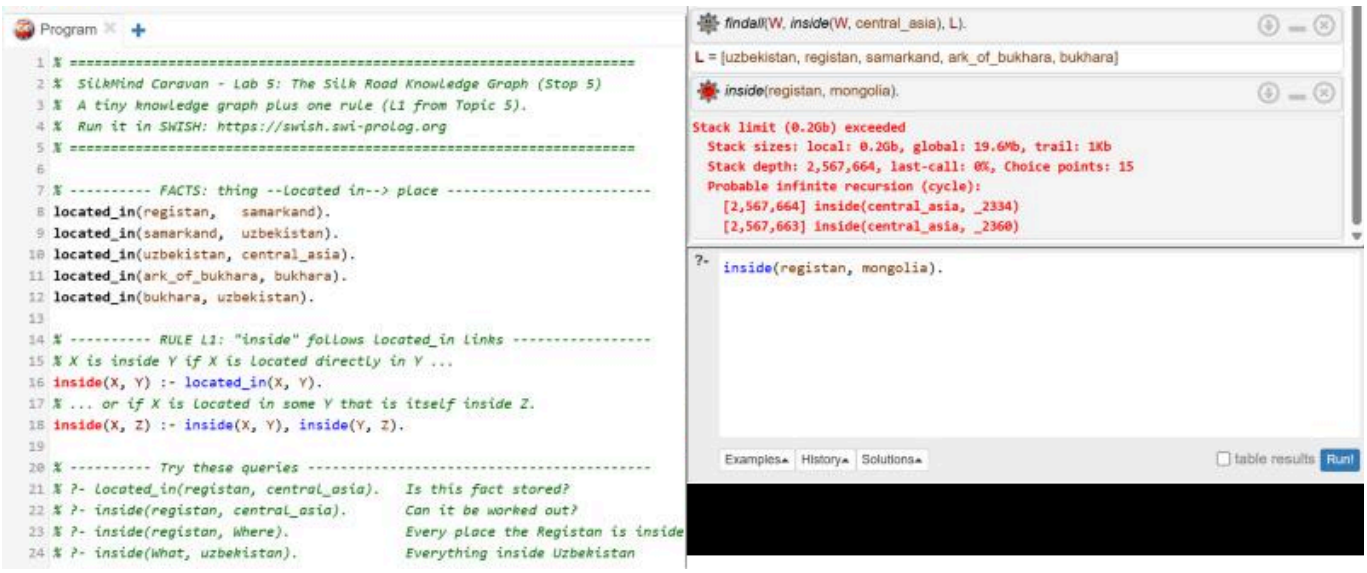


Figure L5.2 Our SWISH run of the broken rule: the extended graph works (top right), but the broken rule makes Prolog recurse until it runs out of memory.

Checkpoint. 1: **Attack**: no rule for Attack matches (health not low, player present), so it stays. 2: age **12**; fix with "12 or over and under 65". 3: Bishkek is located in Central Asia. 4: the symbolic deduction engine.

📮 **POSTCARD FROM THIS STOP**

Greetings from Kashgar! BEK's rules live on in **forms, decision tables, game characters** and **knowledge graphs**, and in **neuro-symbolic** systems where rules and learning work together. One last challenge before the caravan rests...

STOP ★ · THE CARAVANSERAI

The Grand Caravan Challenge



Can you build your own small expert system, and find its weak spots before anyone else does?

Uses everything from Stops 1–5 · ≈ 40 min · Lab in SWISH or on paper



NAVIGATOR NODIRA

Your turn to be the scholar of Bukhara. Build a tiny expert system about something **you** know well.

★ Build, test and break your own expert system

⏱ 40 min

GOAL	Bring together rules (Stop 1), chaining and explanation (Stop 2), the rules-vs-learning choice (Stop 3) and gaps and conflicts (Stops 2 and 4).
PREPARATION	SWISH (or paper). Lab 2A finished. Example solution: `supporting-files/lab-files/finale_tea_expert_example.pl`. Look at it only *after* trying!
TASK	1. Choose a small topic you know: which tea to serve, which bus to take, which plant needs water, which game character to pick… 2. Write **at least 5 rules** (at least two with AND) and facts for **at least 4 test cases**. 3. Include on purpose: one case your rules **cannot** answer (a gap) and one case where **two rules** fire. 4. Run every case in SWISH (or trace on paper) and write the "because…" explanation for one answer.
OBSERVATION	Table: *test case* · *expected answer* · *actual answer* · *rules fired*.
REFLECTION	• What did your gap case show about expert systems? • If you had 10,000 examples of real decisions instead of an expert, would you rather **write rules** or **train a model**? Why? • Which of your rules would be hardest to keep up to date?
FEEDBACK	**Checklist for a strong answer:** ☐ ≥ 5 rules, ≥ 2 using AND ☐ ≥ 4 test cases with expected answers written *before* running ☐ a gap case and a two-rule case ☐ one full explanation chain ☐ a reasoned answer to the rules-or-model question. **Example result** (our tea system, tested in SWISH on 3 Nov 2026): `findall(P-T, recommend(P,T), L)` gave `[aziz-hot_black_tea, lola-mint_tea, timur-ginger_honey_tea, timur-ginger_honey_tea]`. Timur appears **twice** because rules T4 and T5 both fire; Guli is missing because no rule works without a temperature (a gap).

Glossary

One definition per term, the same in this guide, the Markdown content, the slides, the videos and the test bank. "Stop" = where the term is introduced.

Term	Meaning	Stop
AI winter	A period in which funding for and interest in AI research fall sharply, usually after expectations were set too high.	4
Algorithm	A step-by-step procedure that solves a problem in a finite number of clear steps.	1
AND	Boolean operator; true only when **both** parts are true.	1
Backward chaining	Inference that starts from a **goal** and works back to the facts needed to prove it (goal-driven).	2
Boolean logic	Logic in which every statement is true or false, combined with AND, OR and NOT.	1
Brittle	Describes a system that fails badly on cases its rules do not cover.	2
Business rules engine	Software that stores an organisation's decision rules separately from the rest of the program, so they can be changed without rewriting the program.	5
Combinatorial explosion	The very rapid growth in the number of possible combinations as a problem gets larger.	4
Decision table	A table in which each row is one rule: conditions on the left, the result on the right.	5
ELIZA	A 1966 chatbot by Joseph Weizenbaum that replies using keyword pattern rules; it does not understand language.	2 (Lab 2B)
Expert system	A program that uses AI methods, usually a knowledge base of rules and an inference engine, to solve problems in one specialised area.	2
Fact	A statement taken as true for the current case, e.g. has feathers.	2
Feature	A measurable property of an example that an ML program may use, e.g. number of links.	3
Finite state machine	A rule-based design where a character is always in one state and transition rules move it between states.	5
Fire (a rule)	What happens when the inference engine finds a rule's IF part true and adds its THEN part to working memory.	2
Forward chaining	Inference that starts from known facts and keeps firing rules until nothing new can be added (data-driven).	2
Hardware	The physical parts of a computer.	1
Inference	Working out a new fact from facts already known.	2
Inference engine	The program that repeatedly finds rules whose IF part is true and fires them.	2
Knowledge base	The stored collection of rules for one subject area; the same for every case.	2

Knowledge graph	A large collection of facts stored as things connected by named relationships.	5
Label / **labelled data**	The correct answer attached to an example / examples that each have their correct answer.	3
Machine learning (ML)	AI in which software learns patterns from data instead of following rules written in advance.	3
Model	The result of training: the learned "rule" used to answer new cases.	3
Neuro-symbolic AI	A system combining a neural network (learning) with symbolic reasoning (rules).	5
NOT	Boolean operator; flips true to false and false to true.	1
OR	Boolean operator; true when **at least one** part is true (including both).	1
Prediction	A model's answer for a new, unlabelled example.	3
Prolog	A programming language for logic: you write facts and rules, then ask queries; it answers by backward chaining.	2 (Lab 2A)
Pseudocode	Program-like steps in plain language; used to plan an algorithm, cannot be run.	1
Query (Prolog)	A question to a Prolog program, e.g. `is_a(pingu, What).`	2 (Lab 2A)
Rule	An IF–THEN statement: if all conditions (the IF part) are true, the conclusion (the THEN part) is added or carried out.	1–2
Rule conflict	When two rules can both fire and give contradictory conclusions.	2
Software	The instructions (programs) that tell a computer what to do.	1
State / **transition**	What a rule-based character is currently doing / a rule that moves it to another state.	5
Symbol	A name or label that stands for something, e.g. is a bird. The computer does not know its meaning.	2
Symbolic AI	AI that represents knowledge as human-readable symbols and reasons by applying rules to them.	1–2
Threshold rule	A rule that classifies by comparing one value with a cut-off, e.g. links >= 4.	3
Training	The process in which an ML program searches for the pattern that best fits labelled data.	3
Truth table	A table listing every combination of true/false inputs with the result for each.	1
Variable (Prolog)	A word starting with a capital letter (`X`, `What`) that stands for "something" Prolog must find.	2 (Lab 2A)
Working memory	The facts about the current case: starting facts plus facts the engine adds.	2

Appendix A · Printable cards

Print this page and the next on card, cut along the dashed lines. Rule cards are green, fact cards blue, scam/event cards amber.

A1 · Rule cards (Mission 2.1)

R1
IF has feathers
THEN is a bird

R2
IF gives milk
THEN is a mammal

R3
IF is a bird AND cannot fly AND swims
THEN is a penguin

R4
IF is a bird AND can fly AND hunts at night
THEN is an owl

R5
IF is a mammal AND eats meat AND has black stripes
THEN is a tiger

R6
IF is a mammal AND has hooves AND has black stripes
THEN is a zebra

A2 · Fact cards (Mission 2.1): conclusions in the bottom row are added by the "engine"

has feathers

cannot fly

swims

can fly

hunts at night

gives milk

eats meat

has hooves

has black stripes

is a bird

is a mammal

is a penguin

is an owl

is a tiger

is a zebra

A3 · Message cards (Mission 3.1)

Message A
🔗 0 links
label: **not**

Message B
🔗 1 link
label: **not**

Message C
🔗 1 link
label: **not**

Message D
🔗 2 links
label: **not**

Message E
🔗 3 links
label: **not**

Message F
🔗 2 links
label: **scam**

Message G
🔗 4 links
label: **scam**

Message H
🔗 5 links
label: **scam**

Message I
🔗 6 links
label: **scam**

Message J
🔗 8 links
label: **scam**

Message N
🔗 3 links
label: **scam** *(new, for retraining)*

A4 · Event cards (Mission 4.1): years are in the Feedback Corner of Stop 4. Shuffle before playing!

Event 1
Dartmouth summer workshop: the term "artificial intelligence" is used for the project
Year: _____

Event 2
DENDRAL, often called the first expert system, begins at Stanford
Year: _____

Event 3
MYCIN begins: diagnosing blood infections with about 500 rules
Year: _____

Event 4
The Lighthill report criticises AI for the UK Science Research Council
Year: _____

Event 5
DARPA cancels a large speech-recognition grant
Year: _____

Event 6
XCON goes into everyday use at Digital Equipment Corporation
Year: _____

Event 7
Japan's Fifth Generation Computer Systems project starts
Year: _____

Event 8
The LISP-machine market collapses within about a year
Year: _____

Event 9
US Strategic Computing stops new spending on AI
Year: _____

Appendix B · How we tested this guide

The brief asks us to test every activity ourselves and to say which version we used. This is our record for **V1.0**. Team members add their own learner tests below before submission and in later versions.

Activity	Platform / version	Date	What we checked	Result
Lab 2A	SWISH, swish.swi-prolog.org (SWI-Prolog web version live on the test date)	3 Nov 2026	All five queries, giraffe extension, typo, capital-letter variable, "Too many open queries" message	✔ Outputs as printed. Found and fixed: without `:- discontiguous`, adding facts at the end of the file gave confusing warnings; directive added to the starter file.
Lab 5	SWISH (same)	3 Nov 2026	Queries 1–3, Bukhara extension, broken rule	✔ Broken rule gives "Stack limit (0.2Gb) exceeded … Probable infinite recursion".
Grand Challenge example	SWISH (same)	3 Nov 2026	`findall` over all travellers	✔ As printed in the challenge.
Lab 2B	elizabot.js by N. Landsteiner (2005), masswerk.at	3 Nov 2026	7 test inputs	✔ Transcript as printed. Opening line varied between page loads, so replies can vary.
Lab 3	Machine Learning for Kids, "Try it now"	3 Nov 2026	Access route	⚠ Our automated test stopped at the Cloudflare "Verify you are human" box (a person must tick it). **Training steps still to be tested by a team member**; record below.
Missions 2.1, 3.1, Test Bank traces	`supporting-files/examples/*.py`, Python 3	3 Nov 2026	Every trace and mistakes table	✔ Program output matches every answer.
Video links	YouTube IDs checked against video listings	3 Nov 2026	Title, creator, length, chapter times (Crash Course AI #10, CGP Grey)	✔ Team members must watch each recommended part before submission (record below).

Learner tests and member checks (to be completed by Team SilkMind)

Section tried	Tester (did not write it)	Date	Could they complete it? What confused them?	What we changed
Lab 3 (Machine Learning for Kids), full run				
Mission 2.1 (Rule Cards)				
Mission 3.1 (Teach To'ti)				
Videos: watched recommended parts				

Resources, references and acknowledgements

Learning platforms used in labs

- SWISH: SWI-Prolog for SHaring. https://swish.swi-prolog.org (Lab 2A, Lab 5, Grand Challenge)
- Landsteiner, N. (2005). ELIZA (elizabot.js), after Weizenbaum's DOCTOR script. https://www.masswerk.at/elizabot/eliza.html (Lab 2B)
- Lane, D. Machine Learning for Kids. https://machinelearningforkids.co.uk (Lab 3)

Videos and pages recommended

- Josh Darnit. *Exact Instructions Challenge – THIS is why my kids hate me.* YouTube. https://www.youtube.com/watch?v=cDA3_5982h8
- CrashCourse & PBS Digital Studios (2019). *Symbolic AI: Crash Course AI #10.* https://www.youtube.com/watch?v=WHCo4m2VOws
- CGP Grey (2017). *How AI Actually Learns* (first published as "How Machines Learn"). https://www.youtube.com/watch?v=R9OHn5ZF4Uo
- Lighthill, J. (1973). *Artificial Intelligence: A General Survey.* Science Research Council. https://www.aiai.ed.ac.uk/events/lighthill1973/lighthill.pdf
- Google DeepMind (17 Jan 2024). *AlphaGeometry: An Olympiad-level AI system for geometry.* https://deepmind.google/blog/alphageometry-an-olympiad-level-ai-system-for-geometry/

Sources for the explanations

- Encyclopaedia Britannica: *Software*; *Algorithm*; *Boolean algebra*; *Expert system*; *Artificial intelligence: Expert systems*; *Machine learning*.
- BBC Bitesize. *What is software?* · EBSCO Research Starters. *Boolean algebra.*
- OWASP Foundation. *Authentication Cheat Sheet.* · NIST SP 800-63B-4 (2025).
- IBM Developer. *What is machine learning?* · Wikipedia: *Forward chaining*; *AI winter*; *Xcon*.
- McDermott, J. (1981). R1: The formative years. *AI Magazine* 2(2). · McDermott, J. (1982). R1: A rule-based configurer of computer systems. *Artificial Intelligence* 19(1), 39–88.
- Huyck, C. *Configuration with R1/XCon.* Middlesex University course notes.
- IPSJ Computer Museum. *Fifth Generation Computer Systems.* · *Nature* 356, 273 (26 March 1992).
- Hertog, J. *The Birth of Artificial Intelligence.* Dartmouth Alumni Magazine.
- Weizenbaum, J. (1966). ELIZA: a computer program for the study of natural language communication between man and machine. *Communications of the ACM* 9(1), 36–45.
- MDN Web Docs. *Client-side form validation.* · Salesforce Trailhead. *Get to Know Business Rules Engine.*
- *Principles of Computer Game Design and Implementation*, Lecture 23 (University of Liverpool course material).
- Singhal, A. (2012). *Introducing the Knowledge Graph: things, not strings.* Google.

Full links for every source are in the reference section of each Markdown topic file (`content/01`–`05`).

Our own material

The characters BEK, To'ti and Navigator Nodira, the route map, all stories, missions, cards, rule sets, datasets, Prolog and Python files and the diagrams (Figures 1.1–4.1) were created by Team SilkMind. The

Silk Road cities are real places; the caravan stories are fiction and analogies, marked 🐪. Screenshots of SWISH and ELIZA were taken by the team while testing; SWISH and ELIZA belong to their creators.

Team SilkMind and contributions

Member	Role	Contribution to this guide
Afandiyeva Hayat	Team lead	
Khaydarov Sarvarbek	Repository & information lead	
Nabizadehshahrbabak Arsalan	Quality lead	
Egemberdiyeva Aliya	Content lead	
Kenzhebolat Anel	Visual design & video lead	
Mukhtarov Mukhammadshakhruz	Visual design lead	
Bakatov Madi	Video lead	

AI assistance. Drafting of this guide, its activities, the lab files and the test bank was assisted by an AI model (Claude, Anthropic), which also ran the SWISH and ELIZA tests recorded in Appendix B. Details, verification and member decisions are recorded in the project AI-use log (`docs/ai-use-log.md`) and members' personal logs.

Where this guide comes from

Guide section	Markdown Content V2.0 source
Stop 1	`01-software-and-logic.md` §2–§8
Stop 2, Labs 2A/2B	`02-rules-and-inference.md` §2–§9
Stop 3, Lab 3	`03-symbolic-ai-vs-machine-learning.md` §1–§7
Stop 4	`04-history-and-ai-winters.md` §1–§7
Stop 5, Lab 5	`05-symbolic-ai-in-modern-software.md` §1–§7
Glossary	`glossary.md`